

A circular badge with a dashed border and a solid inner circle, containing the word "INDEX" in bold capital letters.

INDEX

A rounded rectangular box with a double border, containing the title "Operating System" in a large, bold, stylized font.

Operating System

1.	Process Management & CPU Scheduling	2.1 – 2.15
2.	Process Synchronization.....	2.16 – 2.30
3.	Deadlocks	2.31 – 2.38
4.	Memory Management	2.39 – 2.51
5.	File System & Disk Management.....	2.52 – 2.58

A decorative footer consisting of several overlapping gray and white geometric shapes, creating a modern, layered effect.

Operating System

Syllabus

System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.

Chapter wise Weightage Analysis

Chapter Paper Year	Ch.1	Ch.2	Ch.3	Ch.4	Ch.5
2008	4	4	4	8	1
2009	0	2	2	4	2
2010	2	3	2	1	0
2011	5	0	0	1	0
2012	3	4	0	2	2
2013	1	5	0	2	0
2014 (P1)	3	0	2	2	1
2014 (P2)	2	2	0	2	1
2014 (P3)	2	0	2	3	0
2015 (P1)	2	2	0	3	3
2015 (P2)	0	0	1	5	2
2015 (P3)	3	2	2	0	0
2016 (P1)	1	2	0	4	2
2016 (P2)	2	4	0	1	0
2017 (P1)	2	2	0	2	0
2017 (P2)	3	0	2	0	1
2018	0	2	3	1	2
2019	3	1	2	2	2
2020	3	2	0	3	2
2021 (P1)	2	2	0	1	0
2021 (P2)	1	4	0	2	0
2022	2	1	1	4	2
2023	3	0	0	4	0

PROCESS MANAGEMENT & CPU SCHEDULING

Introduction & Background

1. [MCQ] [GATE-2011 : 1M]

A computer handles several interrupt sources of which of the following are relevant for this question.

- Interrupt from CPU temperature sensor (raises interrupt if CPU temperature is too high)
- Interrupt from Mouse (raises interrupt if the mouse is moved or a button is pressed)
- Interrupt from Keyboard (raises interrupt when a key is pressed or released)
- Interrupt from Hard Disk (raises interrupt when a disk read is completed)

Which one of these will be handled at the HIGHEST priority?

- Interrupt from Hard Disk
- Interrupt from Mouse
- Interrupt from Keyboard
- Interrupt from CPU temperature sensor

Process concept

2. [MSQ] [GATE-2023 : 1M]

Which one or more of the following need to be saved on a context switch from one thread (T1) of a process to another thread (T2) of the same process?

- Page table base register
- Stack pointer
- Program counter
- General purpose registers

3. [MCQ] [GATE-2020 : 1M]

Consider the following statements about process state transitions for a system using preemptive scheduling.

- A running process can move to ready state.
- A ready process can move to running state.
- A blocked process can move to running state.
- A blocked process can move to ready state.

Which of the above statements are TRUE?

- I, II, III and IV
- II and III only
- I, II and IV only
- I, II and III only

4. [MCQ] [GATE-2015 : 1M]

The maximum number of processes that can be in Ready state for a computer system with n CPUs is

- n
- n^2
- 2^n
- Independent of n

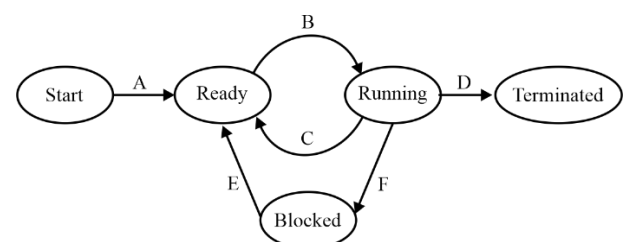
5. [MCQ] [GATE-2011 : 1M]

Let the time taken to switch between user and kernel modes of execution be T_1 while the time taken to switch between two user processes be T_2 . Which of the following is TRUE?

- $T_1 > T_2$
- $T_1 = T_2$
- $T_1 < T_2$
- Nothing can be said about the relation between T_1 and T_2

6. [MCQ] [GATE-2009 : 2M]

In the following process state transition diagram for a uniprocessor system, assume that there are always some process in the ready state:



Now consider the following statements:

- I. If a process makes a transition D, it would result in another process making transition A immediately
 - II. A process P_2 in blocked state can make transition E while another process P_1 is in running state
 - III. The OS uses preemptive scheduling
 - IV. The OS uses non-preemptive scheduling
- Which of the above statements are TRUE?
- (a) I and II
 - (b) I and III
 - (c) II and III
 - (d) II and IV

CPU Scheduling Algorithms

7. [MSQ] [GATE-2023 : 1M]

Which one or more of the following CPU scheduling algorithms can potentially cause starvation?

- (a) First-in First-Out
- (b) Round Robin
- (c) Priority Scheduling
- (d) Shortest Job First

8. [MCQ] [GATE-2022 : 2M]

Consider four processes P, Q, R, and S scheduled on a CPU as per round robin algorithm with a time quantum of 4 units. The processes arrive in the order P, Q, R, S, all at time $t = 0$. There is exactly one context switch from S to Q, exactly one context switch from R to Q, and exactly two context switches from Q to R. There is no context switch from S to P. Switching to a ready process after the termination of another process is also considered a context switch. Which one of the following is NOT possible as CPU burst time (in time units) of these processes?

- (a) $P = 4, Q = 10, R = 6, S = 2$
- (b) $P = 2, Q = 9, R = 5, S = 1$
- (c) $P = 4, Q = 12, R = 5, S = 4$
- (d) $P = 3, Q = 7, R = 7, S = 3$

9. [MSQ] [GATE-2021 : 1M]

Which of the following statement(s) is/are correct in the context of CPU scheduling?

- (a) Turnaround time includes waiting time.
- (b) The goal is to only maximize CPU utilization and minimize throughput.
- (c) Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori.
- (d) Implementing preemptive scheduling needs hardware support.

10. [NAT] [GATE-2021 : 1M]

Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (rounded to nearest integer) is _____ milliseconds.

11. [NAT] [GATE-2020 : 2M]

Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms Shortest Job First (SJF) and Round Robin (RR). For RR, assume that the processes are scheduled in the order P_1, P_2, P_3, P_4 .

Processes	P_1	P_2	P_3	P_4
Burst time (in ms)	8	7	2	4

If the time quantum for RR is 4 ms, then the absolute value of the difference between the average turnaround times (in ms) of SJF and RR (round off to 2 decimal places) is _____.

12. [NAT] [GATE-2019 : 2M]

Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

Process	P_1	P_2	P_3	P_4
Arrival time	0	1	3	4
CPU burst time	3	1	3	Z

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is _____.

13. [NAT] [GATE-2017 : 2M]

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below. None of the process have I/O burst time.

Process	Arrival Time	Burst Time	Priority
P ₁	0	11	2
P ₂	5	28	0
P ₃	12	2	3
P ₄	2	10	1
P ₅	9	16	4

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is ____.

14. [NAT] [GATE-2017 : 1M]

Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	3	3
P ₃	5	5
P ₄	6	2

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is ____ milliseconds.

15. [NAT] [GATE-2016 : 2M]

Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is preemptive shortest remaining-time first.

Process	Arrival Time	Burst Time
P ₁	0	10
P ₂	3	6
P ₃	7	1
P ₄	8	3

The average turn around time of these processes is ____ milliseconds.

16. [MCQ] [GATE-2016 : 1M]

Consider an arbitrary set of CPU-bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue?

- (a) Shortest remaining time first
- (b) Round – robin with time quantum less than the shortest CPU burst
- (c) Uniform random
- (d) Highest priority first with priority proportional to CPU burst length

17. [MCQ] [GATE-2015 : 2M]

For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time?

Process	Arrival Time	Processing Time
A	0	3
B	1	6
C	4	4
D	6	2

- (a) First come First serve.
- (b) Non-preemptive shortest job first.
- (c) Shortest Remaining time.
- (d) Round Robin with Quantum value two.

18. [NAT] [GATE-2015 : 2M]

Consider a uniprocessor system executing three tasks T₁, T₂ and T₃, each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of 3, 7 and 20 milliseconds,

respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, which is the highest priority task scheduled first. Each instance of T_1 , T_2 and T_3 , requires an execution time of 1, 2 and 4 milliseconds, respectively. Given that all task initially arrive at the beginning of the 1st millisecond and task preemptions are allowed, the first instance of T_3 completes its execution at the end of ____ milliseconds.

19. [NAT] [GATE-2014 : 2M]

An operating system uses shortest remaining time first scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds):

Process	Arrival Time	Burst Time
P_1	0	12
P_2	2	4
P_3	3	6
P_4	8	5

The average waiting time (in milliseconds) of the processes is ____.

20. [NAT] [GATE-2014 : 2M]

Three processes A, B and C each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires t_c CPU milliseconds and then initiates a single I/O operation that lasts for t_{io} milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

Process id	t_c	t_{io}
A	100 ms	500 ms
B	350 ms	500 ms
C	200 ms	500 ms

The process A, B and C are started at times 0, 5 and 10 milliseconds respectively, in a pure time-sharing system (round robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process C would complete its first I/O operation is ____.

21. [NAT] [GATE-2014 : 2M]

Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds.

Process Name	Arrival Time	Execution Time
A	0	6
B	3	2
C	5	4
D	7	6
E	10	3

Using the shortest remaining time first scheduling algorithm, the average process turnaround time (in msec) is ____

22. [MCQ] [GATE-2013 : 1M]

A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with priority zero (the lowest priority). The scheduler re-evaluates the process priorities every T time units and decides the next process to schedule. Which one of the following is TRUE if the processes have no I/O operations and all arrive at time zero?

- (a) This algorithm is equivalent to the first-come first-serve algorithm
- (b) This algorithm is equivalent to the round-robin algorithm
- (c) This algorithm is equivalent to the shortest-job-first algorithm
- (d) This algorithm is equivalent to the shortest-remaining-time-first algorithm

23. [MCQ] [GATE-2012 : 2M]
Consider the 3 processes, P₁, P₂ and P₃ shown in table.

Process	Arrival Time	Time Units Required
P ₁	0	5
P ₂	1	7
P ₃	3	4

The completion order of the 3 processes under the policies FCFS and RR2 (round robin scheduling with CPU quantum of 2-time units) are:

- (a) FCFS: P₁, P₂, P₃, RR2: P₁, P₂, P₃
(b) FCFS: P₁, P₃, P₂, RR2: P₁, P₃, P₂
(c) FCFS: P₁, P₂, P₃, RR2: P₁, P₃, P₂
(d) FCFS: P₁, P₃, P₂, RR2: P₁, P₂, P₃

24. [MCQ] [GATE-2011 : 2M]
Consider the following table of arrival time and burst time for three processes P₀, P₁ and P₂.

Process	Arrival Time	Burst Time
P ₀	0 ms	9 ms
P ₁	1 ms	4 ms
P ₂	2 ms	9 ms

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

- (a) 5.0 ms (b) 4.33 ms
(c) 6.33 ms (d) 7.33 ms

25. [MCQ] [GATE-2010 : 1M]
Which of the following statements are true?
I. Shortest remaining time first scheduling may cause starvation.
II. Preemptive scheduling may cause starvation.
III. Round robin is better than FCFS in terms of response time.
(a) I only (b) I and III only
(c) II and III only (d) I, II and III

26. [MCQ] [GATE-2008 : 2M]
If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will
(a) degenerate to shortest job first
(b) degenerate to priority scheduling
(c) degenerate to first come first serve
(d) None of the above

System Call

27. [MSQ] [GATE-2023 : 1M]
Which one or more of the following options guarantee that a computer system will transition from user mode to kernel mode?
(a) Function Call (b) Malloc Call
(c) Page Fault (d) System Call

28. [MSQ] [GATE-2021: 1M]
Which of the following standard C library functions will always invoke a system call when executed from a single-threaded process in a UNIX/Linux operating system?
(a) exit (b) malloc
(c) sleep (d) strlen

29. [NAT] [GATE-2019 : 1M]
The following C program is executed on a Unix/Linux system:

```
#include<unistd.h>
int main( )
{
    int i;
    for (i=0;i<10;i++)
        if (i%2==0)
            fork( );
    return 0;
}
```

The total number of child processes created is _____.

30. [MCQ] [GATE-2012 : 1M]

A process executes the code

```
fork ();
fork ();
fork ();
```

The total number of child processes created is

- (a) 3 (b) 4
(c) 7 (d) 8

31. [MCQ] [GATE-2008 : 2M]

A process executes the following code

```
for (i = 0; i < n; i++) fork ();
```

The total number of child processes created is

- (a) n (b) $2^n - 1$
(c) 2^n (d) $2^{n+1} - 1$

Threads

32. [MCQ] [GATE-2017 : 1M]

Which of the following is/are shared by all the threads in a process?

- | | |
|--------------------|---------------------|
| I. Program counter | II. Stack |
| III. Address space | IV. Registers |
| (a) I and II only | (b) III only |
| (c) IV only | (d) III and IV only |

33. [MCQ] [GATE-2017 : 1M]

Threads of a process share

- (a) global variables but not heap
(b) heap but not global variables
(c) neither global variables nor heap
(d) both heap and global variables

34. [MCQ] [GATE-2014 : 1M]

Which of the following is FALSE?

- (a) User level thread are not scheduled by the kernel.
(b) When a user level thread is blocked, all other threads of its process are blocked.
(c) Context switching between user level threads is faster than context switching between kernel level threads.
(d) Kernel level threads cannot share the code segment.

35. [MCQ] [GATE-2011 : 1M]

A thread is usually defined as a “light weight process” because an Operating System (OS) maintains smaller data structures for a thread than for a process. In relation to this, which of the following is TRUE?

- (a) On per-thread basis, the OS maintains only CPU register state.
(b) The OS does not maintain a separate stack for each thread.
(c) On per-thread basis, the OS does not maintain virtual memory state.
(d) On per-thread basis, the OS maintains only scheduling and accounting information.

□□□

ANSWER KEY

- | | | | |
|------------------|----------------|----------------------|--------------------|
| 1. (d) | 2. (b, c, d) | 3. (c) | 4. (d) |
| 5. (c) | 6. (c) | 7. (a, c, d or c, d) | 8. (d) |
| 9. (a, c, d) | 10. (12 to 12) | 11. (5.25 to 5.25) | 12. (2 to 2) |
| 13. (29 to 29) | 14. (3 to 3) | 15. (8.2 to 8.3) | 16. (a) |
| 17. (c) | 18. (12 to 12) | 19. (5.5 to 5.5) | 20. (1000 to 1000) |
| 21. (7.2 to 7.2) | 22. (b) | 23. (c) | 24. (a) |
| 25. (d) | 26. (c) | 27. (c, d) | 28. (a, c) |
| 29. (31 to 31) | 30. (c) | 31. (b) | 32. (b) |
| 33. (d) | 34. (d) | 35. (c) | |

SOLUTIONS

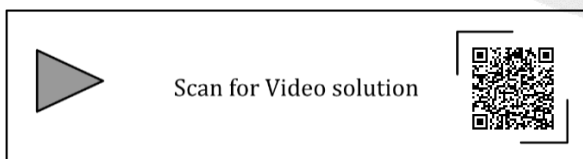
1. (d)

Interrupt from CPU temperature sensor should be given highest priority. Delaying a CPU temperature sensor could result in major consequences, overheated CPU can damage internal circuits, and hence hampering entire computer system.

The priority of above events is given as:

CPU temperature sensor > Hard disk > Mouse > Keyboard.

Therefore, option d is the correct answer.



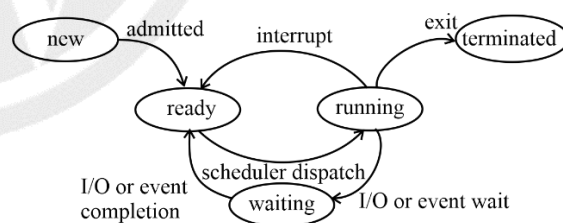
2. (b, c, d)

Page table base register is used to access the virtual address space, as all threads share the same address space, so there is no need to save it. Therefore, option A is incorrect.

Every thread has its own stack pointer, program counter, and general purpose registers, so we need to save them. Therefore, Option B, C, and D are correct.

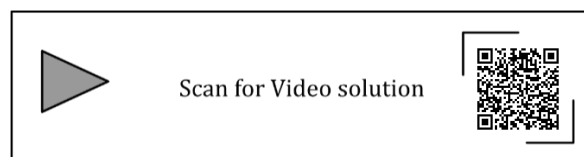
3. (c)

A process state diagram with pre-emptive scheduling is as follows:



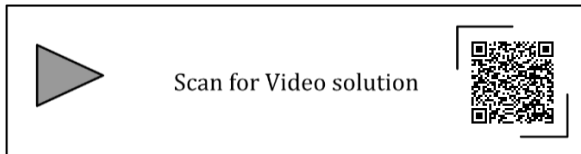
So, from the given diagram we can say

- I. A running process can move to ready state.
 - II. A ready process can move to running state.
 - IV. A blocked process can move to ready state.
- Therefore, option C is correct.



4. (d)

The maximum number of processes present in the ready state is independent of number of CPU's the system have. Number of processes in ready state depends upon the size of ready queue. Therefore, option D is the correct answer.



5. (c)

Given,

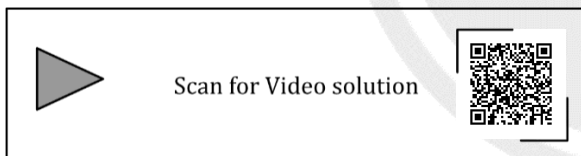
User Mode $\rightarrow$ Kernel Mode = T_1

User process 1 $\rightarrow$ User process 2 = T_2 [Context Switching]

Context switching includes mode shifting, first preempt the process, shift the mode from user to kernel, OS will save and load the PCB, again shift the mode from kernel to user.

From this we can say context switching time is superset and mode shifting is a subset. Therefore, T_2 is greater than T_1 .

Hence, option C is the correct answer.



6. (c)

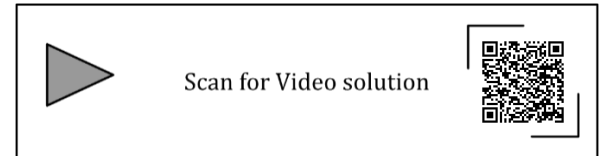
If a process is completed it is not necessary that you should load a new process into memory. Therefore, statement I is false.

A process P_2 in blocked state can make transition E while another process P_1 is in running state. TRUE. Process moving from block to ready state and another process is in running state, this situation is possible and hence statement II is true.

The OS uses preemptive scheduling. TRUE. The given diagram makes a transition from running to ready state and hence this indicates that processes

can be preempted while running. So, statement III is also true and statement IV is false.

Hence, option C is the correct answer.



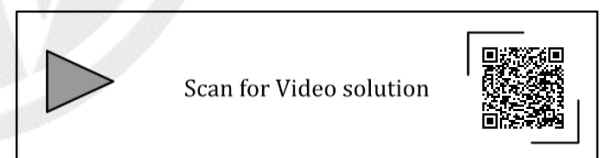
7. (a, c, d, or c, d)

First-In-First-Out may suffer from starvation in some exceptional case. In FIFO if a process has arrived then it will definitely get CPU once all the processes arrived before that are serviced. But processes may starve when a process has very long burst time and wait in waiting queue.

Round robin never suffers from starvation because every process gets time to execute on CPU in circular manner.

Priority Scheduling suffers from starvation as processes with lower priority may starve for processor if higher priority process keeps arriving. Shortest Job First also suffers from starvation as processes with higher burst time may starve for processor when processes with lower burst time keeps on arriving.

Therefore, option C and D are correct



8. (d)

Given,

1. Switching to ready processes after termination of currently executing process also a context switch.
2. There is no context switch from S to P. (Therefore P should be completed within 1 TQ)
3. There are exactly two context switches from Q to R (Therefore Q and R both should requires more than 1 TQ)

4. There is exactly one context switch from R to Q
(Therefore Q require more time than R)
5. Exactly one context switch from R to S.
(Therefore S should be complete within 1 TQ
otherwise another CS from R to S required.)
6. Exactly one context switch from S to Q.

$S \rightarrow Q = 1$

$R \rightarrow Q = 1$

$Q \rightarrow R = 2$

$S \rightarrow P = \text{No context switch}$

From the given points, we can conclude:

1. Burst Time (P) ≤ 4

2. Burst Time (Q) > 8

3. Burst Time (S) ≤ 4

$\rightarrow$ Round Robin (Time Quantum = 4)

R.Q:

P	Q	R	S	Q	R	Q
---	---	---	---	---	---	---

CPU

P	Q	R	S	Q	R	Q
---	---	---	---	---	---	---

0

We have said that Burst Time (Q) > 8 . Only option D does not satisfy this situation and hence it is NOT possible as CPU bursts.



Scan for Video solution



9. (a, c, d)

- (a) Turnaround time includes waiting time. CORRECT. As, Turnaround time = waiting time + burst time.
- (b) The goal is to only maximize CPU utilization and minimize throughput. INCORRECT. The goal of CPU scheduling is to increase the CPU utilization and throughput, and minimise the response time.

(c) Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori. CORRECT. Round robin policy works irrespective of burst time of the process, it gives equal amount of time quantum to each process on processor.

(d) Implementing preemptive scheduling needs hardware support. CORRECT. To preempt a process, hardware support is required, a timer interrupt is generated whenever time quantum expires and the interrupted process is enqueued in ready queue and another process is scheduled to the CPU.



Scan for Video solution



10. (12)

It is given that we have to find out the minimum achievable average waiting time for these three processes in a non-preemptive scheduler. This is always possible with Shortest Job First CPU scheduling algorithm. Hence, we will be using SJF to solve this problem.

Gantt chart using SJF:

P ₃	P ₁	P ₂
0	10	26 46

$$AV.W.T = \frac{10 + 26 + 0}{3}$$

$$= \frac{36}{3} = 12$$

Therefore, 12 is the correct answer.



Scan for Video solution



11. (5.25)

P.NO	Arrive Time	Burst Time
P ₁	0	8
P ₂	0	7
P ₃	0	2
P ₄	0	4

Gantt chart using SJF:

CPU	<table><tr><td>P₃</td><td>P₄</td><td>P₂</td><td>P₁</td></tr></table>					P ₃	P ₄	P ₂	P ₁
P ₃	P ₄	P ₂	P ₁						
	0	2	6	13	21				
	Time								

TAT = Completion time - arrival time

$$\text{TAT } P_1 = 21 - 0 = 21$$

$$\text{TAT } P_2 = 13 - 0 = 13$$

$$\text{TAT } P_3 = 2 - 0 = 2$$

$$\text{TAT } P_4 = 6 - 0 = 6$$

$$\text{Average TAT} = \frac{42}{4} = 10.5$$

Round RobinReady Queue- $[P_1]; [P_2]; [P_3]; [P_4]; [P_1]; [P_2];$

Gantt chart using round robin:

CPU	P ₁	P ₂	P ₃	P ₄	P ₁	P ₂	
	0	4	8	10	14	18	21

TAT

$$\text{TAT } P_1 = 18$$

$$\text{TAT } P_2 = 21$$

$$\text{TAT } P_3 = 10$$

$$\text{TAT } P_4 = 14$$

$$\text{Total} = 63$$

$$\text{Average TAT} = \frac{63}{4} = 15.75$$

$$\text{Difference} = [15.75 - 10.5] = 5.25$$



Scan for Video solution



12. (2 ms)

P.NO	Arrive Time	Burst Time
P ₁	0	3
P ₂	1	1
P ₃	3	3
P ₄	4	Z

Average Waiting Time = 1 ms

CASE I: If $Z < 3$

Gantt chart using SRTF:

P ₁	P ₂	P ₁	P ₄	P ₃	
0	1	2	4	4 + Z	7 + Z

P.No	Arrival Time	Burst Time	Priority	Completion Time	Turnaround Time	Waiting Time
P ₁	0	11	2	49	49	38
P ₂	5	28	0 (Highest)	33	28	0
P ₃	12	2	3	51	39	37
P ₄	2	10	1	40	38	28
P ₅	9	16	4 (Lowest)	67	58	42

$$\text{Average waiting Time} = \frac{1 + 0 + (z + 1)}{4}$$

$$= \frac{z + 2}{4} + 1 \text{ ms}$$

$$\Rightarrow \frac{z + 2}{4} + 1 \Rightarrow Z + 2 = 4 \Rightarrow Z + 4 - 2 = 2$$

$$\Rightarrow Z = 2 \text{ ms}$$

CASE II: If $Z > 3$

Average Waiting Time > 1 ms

CPU	P ₁	P ₂	P ₁	P ₃	P ₄	
	0	1	2	4	7	7 + Z

$$\text{Average waiting Time} = \frac{1 + 0 + 1 + 3}{4} = 1 \text{ ms}$$

$$\Rightarrow 5/4 \neq 1$$

$$\Rightarrow 1.25 \neq 1$$

So, the value of Z could not be greater than 3, and hence the correct answer is 2.



Scan for Video solution



13. (29)

P.No	Arrival Time	Burst Time	Priority
P ₁	0	11	2
P ₂	5	28	0 (Highest)
P ₃	12	2	3
P ₄	2	10	1
P ₅	9	16	4 (Lowest)

Gantt chart using Pre-emptive priority scheduling algorithm:

P ₁	P ₄	P ₂	P ₄	P ₁	P ₃	P ₅
0	2	5	33	40	49	51 67

$$\text{Average Waiting Time} = \frac{38 + 0 + 37 + 28 + 42}{5} = \frac{145}{5} = 29$$



Scan for Video solution



14. (3.0)

P.NO	Arrive Time	Burst Time
P ₁	0	7
P ₂	3	3
P ₃	5	5
P ₄	6	2

Gantt chart using SRTF scheduling algorithm:

	P ₁	P ₂	P ₄	P ₁	P ₃
	0	3	6	8	12 17
P.NO	Arrival Time	Burst Time	Completion time	Turnaround time	Waiting time
P ₁	0	7	12	12	5
P ₂	3	3	6	3	0
P ₃	5	5	17	12	7
P ₄	6	2	8	2	0

$$\text{AV.W.T} = \frac{5 + 0 + 7 + 0}{4} = \frac{12}{4}$$

Average Waiting Time = 3.0



Scan for Video solution



15. (8.25)

Gantt chart using SRTF:

	P ₁	P ₂	P ₃	P ₂	P ₄	P ₁	
	0	3	7	8	10	13	20
Process	Arrival Time	Burst Time	Completion Time	Turnaround Time			
P ₁	0	10	20	20			
P ₂	3	6	10	7			
P ₃	7	1	8	1			
P ₄	8	3	13	5			

$$\text{AV.TAT} = \frac{20 + 7 + 1 + 5}{4} = \frac{33}{4} = 8.25$$



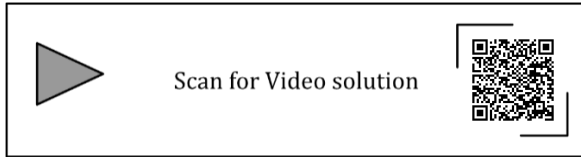
Scan for Video solution



16. (a)

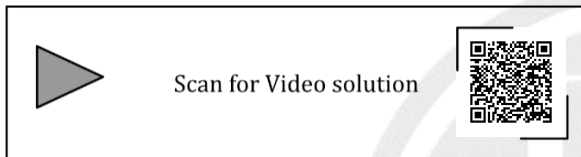
Shortest job first (SJF) minimizes the waiting time, SJF gives the minimum average waiting time as compared with other algorithms. Here it is given that arrival time of all processes is same, so SRTF acts same as SJF.

Therefore, option A is correct.



17. (c)

Shortest remaining time first or Shortest job first will always generate minimum average turnaround time and average waiting time. Here also preemptive SJF or SRTF will always give minimum average waiting time as compared to non-preemptive SJF. Therefore, option C is the correct answer.



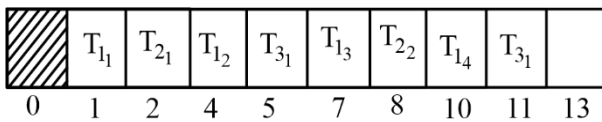
18. (12)

Prio	Period	P.NO	A.T	B.T	Insta
1/3	3	T1	1	1	1; 4; 7; 10; 13; 16...
1/7	7	T2	1	2	1; 8; 15; ...
1/20	4	T3	1	4	1; 21; 41; ...

Pre-Prio:

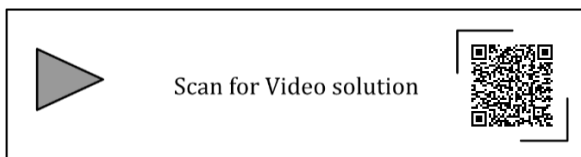
R.O $T_{1_1}; T_{2_1}; T_{3_1}; T_{1_2}; T_{1_3}; T_{2_2}; T_{1_4};$

Gantt Chart:



So, at the end of 12 millisecond and at the beginning of 13 millisecond the first instance of T3 completed its execution.

Therefore, 12 is the correct answer.



19. (5.5)

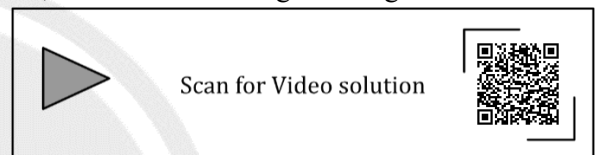
Gantt Chart using SRTF:

	P ₁	P ₂	P ₃	P ₄	P ₁
0	2	6	12	17	27

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P ₁	0	12	27	27	15
P ₂	2	4	6	4	0
P ₃	3	6	12	9	3
P ₄	8	5	17	9	4

$$AV.W.T = \frac{15 + 0 + 3 + 4}{4} = \frac{22}{4} = 5.5$$

So, 5.5 ms is the average waiting time.



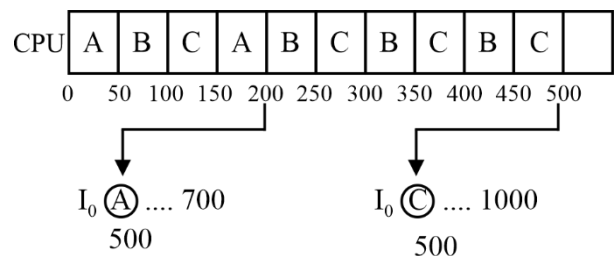
20. (1000)

Given,

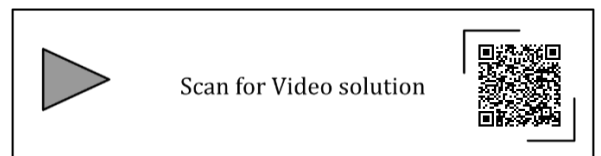
Time Quantum = 50

P.No	Arrival Time	Burst Time	IOBT
A	0	100	500
B	5	350	500
C	10	200	500

Gantt Chart:



C has completed its IO at time 1000 ms.



21. (7.2)

Gantt Chart using SRTF:

A	B	A	C	E	D	
0	3	5	8	12	15	21

Process Name	Arrival Time	Burst Time	Completion Time	Turnaround Time
A	0	6	8	8
B	3	2	5	2
C	5	4	12	7
D	7	6	21	14
E	10	3	15	5

$$AV.TAT = \frac{8+2+7+14+5}{5}$$

$$AV.TAT = \frac{36}{5} = 7.2$$



Scan for Video solution



22. (b)

This algorithm will behave as preemptive FCFS and is equivalent to the round-robin algorithm. Therefore, option B is the correct answer.



Scan for Video solution



23. (c)

In FCFS all the process's gets executed as per their time of arrival, so using FCFS processes will be completed as P1, P2, P3

Now, checking the completion sequence using round-robin scheduling:

P.No	Arrival Time	Burst Time
1	0	5
2	1	7
3	3	4

Round robin Time Quantum = 2

Ready Queue-

 $P_1; P_2; P_1; P_3; P_2; P_1; P_3; P_2$

Gantt Chart:

P ₁	P ₂	P ₁	P ₃	P ₂	P ₁	P ₃	P ₂	
0	2	4	6	8	10	11	13	16

Here, Process are completed as P1, P3, P2.

So, option C is the correct answer.



Scan for Video solution



24. (a)

Gantt Chart Preemptive SJF:

CPU	P ₀	P ₁	P ₀	P ₂	
	0	1	5	13	22

P. No	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P ₀	0	9	13	13	4
P ₁	1	4	5	4	0
P ₂	2	9	22	20	11

$$AV.W.T = \frac{4+0+11}{3} = \frac{15}{3} = 5$$

Hence, option A is the correct answer.



Scan for Video solution

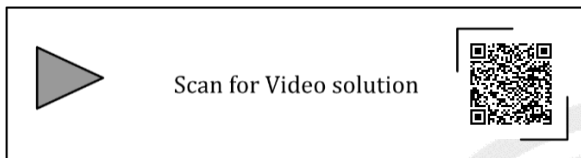


25. (d)

I. Shortest remaining time first scheduling may cause starvation. TRUE. Processes with longer burst time may starve if processes with shorter burst time keeps on arriving.

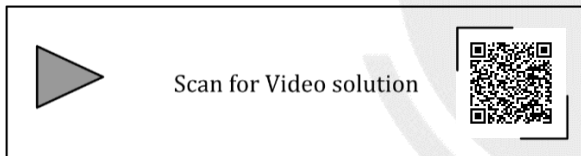
- II.** Preemptive scheduling may cause starvation.
TRUE. Preemptive scheduling may cause starvation for example SRTF is preemptive scheduling and it can cause starvation. So this statement is also correct.
- III.** Round robin is better than FCFS in terms of response time. TRUE. Round robin has defined time quantum for each process, therefore it has better response time than FCFS.

Hence, all statements are true and option D is correct.



26. (c)

If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will degenerate to first come first serve.

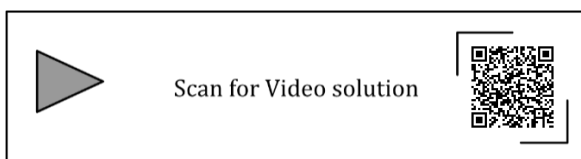


27. (c, d)

A computer system has to make a transition from user mode to kernel mode, whenever a page fault occurs, the page fault handler will run in kernel mode.

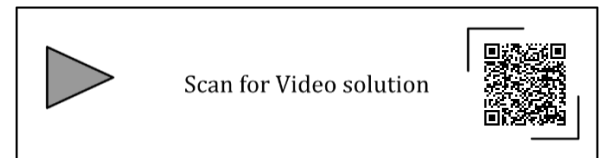
System call always makes a transition from user to kernel mode.

Therefore, option C and D are correct.



28. (a, c)

- (a) exit will always invoke a system call.
(b) malloc will not invoke a system call always, it may or may not invoke a system call.
(c) sleep will always invoke a system call.
(d) strlen will never invoke a system call.

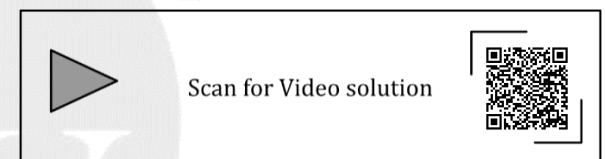


29. (31)

We know that, n-forks creates $(2^n - 1)$ child processes

$$n = 10; (n \% 2 = 0) \Rightarrow 5 \text{ times}$$

$$\therefore \text{No of child processes} = 2^5 - 1 = 31$$



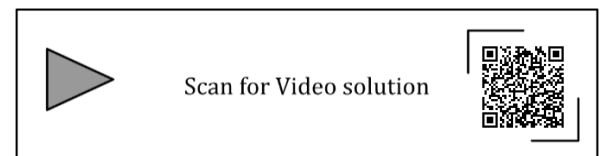
30. (c)

For, n-fork system call $\Rightarrow$ Total child processes = $2^n - 1$

$$\text{Here, } n = 3$$

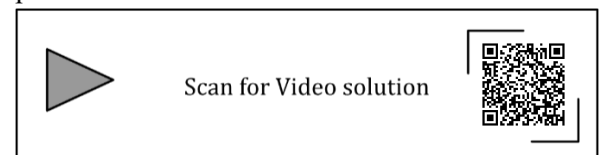
$$\text{So, } 2^3 - 1 = 7$$

So, option C is the correct answer.



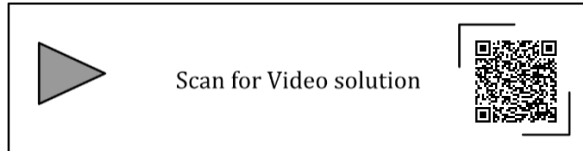
31. (b)

For, n fork() system calls, total number of child processes created is $2^n - 1$.



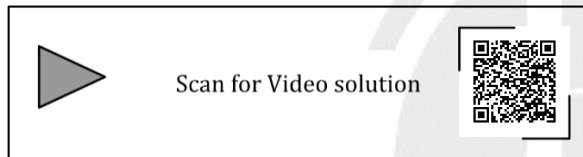
32. (b)

Thread is the light weight process and each thread comprises its own stack, program counter, register. Only the address space of the thread is shared among threads of a single process. So, option B is the correct answer.



33. (d)

A thread of a process shares with other threads are: Data section, Code section, Address space, permissions, etc. So, option D is the correct answer. Threads of a process shares both heap and global variables.



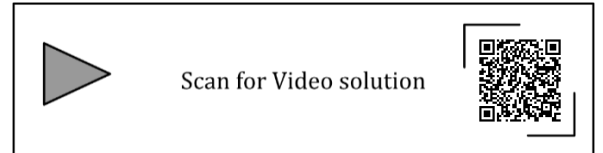
34. (d)

User level threads are scheduled by the thread library not by kernel. Kernel even don't know about the existence of user level threads. So, A is correct. When a user level thread is blocked, all other thread of its process are blocked. Option B is also correct. Context switching between user level threads is faster than kernel level thread because user level threads have to store nothing, whereas kernel level

thread save and restore registers, program counter, etc. Therefore, option C is also correct.

Kernel level threads cannot share the code segment. Threads can share the code segment, threads have separate address space but they can share same code segment.

So, option D is incorrect.



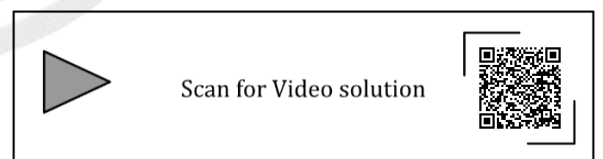
35. (c)

On per-thread basis, the OS maintains only CPU register. **INCORRECT**. As it also maintains thread state, thread ID, thread priority, etc. are also maintained.

The OS maintains a separate stack for each thread, so option B is also **INCORRECT**.

On per-thread basis, the OS does not maintain virtual memory state. **CORRECT**. Virtual memory state is actually address space, and it is not unique for each thread, address space is shared among multiple threads.

On per-thread basis, the OS maintains only scheduling and accounting information. **INCORRECT**. As thread maintains other data also like thread priority, thread state, etc.



Critical Section Problem**1. [MCQ] [GATE-2016 : 2M]**

Consider the following proposed solution for the critical section problem. There are n processes: $P_0 \dots P_{n-1}$. In the code, function pmax returns an integer not smaller than any of its arguments. For all i , $t[i]$ is initialized to zero.

Code for P_i

```
do
{
c[i] = 1; t[i] = pmax(t[0], ..., t[n-1]) + 1; c[i] = 0;
for every j in {0, ..., n-1}
{
while (c[j]);
while (t[j] != 0 && t[j] <= t[i]);
}
```

Critical Section; $t[i] = 0$;

Remainder Section;

} while (true);

which one of the following is TRUE about the above solution?

- (a) At most one process can be in the critical section at any time.
- (b) The bounded wait condition is satisfied.
- (c) the progress condition is satisfied.
- (d) It cannot cause a deadlock.

2. [MCQ] [GATE-2016 : 2M]

Consider the following two-process synchronization solution

Process 0

```
Entry: loop while (turn == 1);
(critical section)
Exit: turn = 1;
```

Process 1

```
Entry: loop while (turn == 0);
(critical section)
```

```
Exit: turn = 0;
```

The shared variable turn is initialized to zero.

Which one of the following is TRUE?

- (a) This is a correct two-process synchronization solution.
- (b) This solution violates mutual exclusion requirement.
- (c) This solution violates progress requirement.
- (d) This solution violates bounded wait requirement.

3. [MCQ] [GATE-2015 : 2M]

Two processes X and Y need to access a critical section. Consider the following synchronization construct used by both the processes.

Process X

```
/* other code for process X */
```

```
while (true)
```

```
{
varP = true;
while (varQ == true)
{ /* Critical Section */
varP = false;
}
```

```
}
/* other code for process X */
```

Process Y

```
/* other code for process Y */
```

```
while (true)
```

```
{
varQ = true;
while (varP == true)
```

```

{ /* Critical Section */
    varQ = false;
}
}
/* other code for the process Y */

```

Here, varP and varQ are shared variables and both are initialized to false. Which one of the following statements is true?

- The proposed solution prevents deadlock but fails to guarantee mutual exclusion.
- The proposed solution guarantees mutual exclusion but fails to prevent deadlock
- The proposed solution guarantees mutual exclusion and prevents deadlock
- The proposed solution fails to prevent deadlock and fails to guarantee mutual exclusion

Hardware Solution (TSL, etc)

4. [MCQ] [GATE-2012 : 2M]

Fetch_And_Add (X, i) is an atomic Read-Modify-Write instruction that reads the value of memory location X, increments it by the value i, and returns the old value of X, it is used in the pseudocode shown below to implement a busy-wait lock. L is unsigned integer shared variable initialized to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

```

AcquireLock (L) {
    while (Fetch_And_Add (L, 1))
        L = 1;
}
ReleaseLock (L) {
    L = 0;
}

```

This implementation

- fails as L can overflow.
- fails as L can take on a non-zero value when the lock is actually available.
- works correctly but may starve some processes
- works correctly without starvation.

5. [MCQ] [GATE-2009 : 2M]

The enter_CS () and leave_CS () functions to implement critical section of a process are realized using test-and-set instruction as follows:

```

void enter_CS(X)
{
    while (test-and-set(X));
}

void leave_CS(X)
{
    X = 0;
}

```

In the above solution, X is a memory location associated with the CS and is initialized to 0.

Now consider the following statements

- The above solution to CS problem is deadlock free.
- The solution is starvation free.
- The processes enter CS in FIFO order.
- More than one process can enter CS at the same time.

Which of the above statements are TRUE?

- I only
- I and II
- II and III
- IV only

Software Solution

6. [MSQ] [GATE-2021 : 2M]

Consider the following multi-threaded code segment (in a mix of C and pseudo-code), invoked by two processes P₁ and P₂, and each of the processes spawns two threads T₁ and T₂.

```

int x = 0; // global
Lock L1; // global
main() {
    create a thread to execute foo(); // Thread T1
    create a thread to execute foo(); // Thread T2
    wait for the two threads to finish execution;
    print (x);}
foo() {
    int y = 0;
    Acquire L1;
    x = x + 1;
    y = y + 1;
    Release L1;
}

```

```
print (y); }
```

Which of the following statements(s) is/are correct?

- (a) Both P_1 and P_2 will print the value of x as 2.
- (b) At least one of P_1 and P_2 will print the value of x as 4.
- (c) At least one of the threads will print the value of y as 2.
- (d) Both T_1 and T_2 , in both the processes, will print the value of y as 1.

7. [MCQ] [GATE-2017 : 2M]

A multithreaded program P executes with x number of threads and uses y number of locks for ensuring mutual exclusion while operating on shared memory locations. All locks in the program are non-reentrant, i.e., if a thread holds a lock l , then it cannot re-acquire lock l without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The minimum value of x and the minimum value of y together for which execution of P can result in a deadlock are:

- (a) $x = 1, y = 2$ (b) $x = 2, y = 1$
- (c) $x = 2, y = 2$ (d) $x = 1, y = 1$

8. [MCQ] [GATE-2010 : 1M]

Consider the methods used by processes P_1 and P_2 for accessing their critical sections whenever needed, as given below. The initial values of shared boolean variables S_1 and S_2 are randomly assigned.

Method used by P_1	Method used by P_2
while ($S_1 == S_2$); Critical Section $S_1 = S_2$;	While($S_1 != S_2$) Critical Section $S_2 = \text{not}(S_1)$;

Which one of the following statements describes the properties achieved?

- (a) Mutual exclusion but not progress
- (b) Progress but not mutual exclusion
- (c) Neither mutual exclusion nor progress
- (d) Both mutual exclusion and progress

9. [MSQ] [GATE-2021-CS: 2M]

Consider a computer system with multiple shared resource types, with one instance per resource type. Each instance can be owned by only one process at a time. Owning and freeing of resources are done by holding a global lock (L). The following scheme is used to own a resource instance:

function OWNRESOURCE(Resource R)

Acquire lock L // a global lock

if R is available then

Acquire R

Release lock L

else

if R is owned by another process P then

Terminate P , after releasing all resources owned by P

Acquire R

Restart P

Release lock L

end if

end if

end function

Which of the following choice(s) about the above scheme is/are correct?

- (a) The scheme ensures that deadlocks will not occur.
- (b) The scheme may lead to live-lock.
- (c) The scheme may lead to starvation.
- (d) The scheme violates the mutual exclusion property.

OS Based Solution

10. [MSQ] [GATE-2021 : 2M]

Consider the following pseudocode, where S is a semaphore initialized to 5 in line # 2 and counter is a shared variable initialized to 0 in line # 1. Assume that the increment operation in line # 7 is not atomic.

1. int counter = 0;
2. Semaphore $S = \text{init}(5)$;
3. void parop (void)
4. {
5. wait (S);
6. wait (S);
7. counter++;

8. signal (S);
9. signal (S);
10. }

If five threads execute the function parop concurrently, which of the following program behavior(s) is/are possible?

- (a) The value of counter is 5 after all the threads successfully complete the execution of parop
- (b) The value of counter is 1 after all the threads successfully complete the execution of parop
- (c) The value of counter is 0 after all the threads successfully complete the execution of parop
- (d) There is a deadlock involving all the threads

11. [MCQ] [GATE-2020 : 2M]

Each of a set of n processes executes the following code using two semaphores a and b initialized to 1 and 0, respectively. Assume that count is a shared variable initialized to 0 and not used in CODE SECTION P.

CODE SECTION P

```
wait (a); count = count + 1;
if (count == n) signal (b);
signal (a); wait (b); signal (b);
```

CODE SECTION Q

What does the code achieve?

- (a) It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P.
- (b) It ensures that all processes execute CODE SECTION P mutually exclusively.
- (c) It ensures that two processes are in CODE SECTION Q at any time.
- (d) It ensures that at most $n-1$ process are in CODE SECTION P at any time.

12. [NAT] [GATE-2016 : 2M]

Consider a non-negative counting semaphore S . The operation $P(S)$ decrements S , and $V(S)$ increments S . During an execution, 20 $P(S)$ operations and 12 $V(S)$

operations are issued in some order. The largest initial value of S for which at least one $P(S)$ operation will remain blocked is ____

13. [MCQ] [GATE-2014 : 1M]

Consider the procedure below for the Producer-Consumer problem which uses semaphores:

semaphore $n = 0$;

semaphore $s = 1$;

Void producer ()

```
{
    while (true)
    {
        produce ();
        semWait(s);
        addToBuffer ();
        semSignal(s);
        semSignal(n);
    }
}
```

Void consumer ()

```
{
    while(true)
    {
        semWait(s);
        semWait(n);
        removeFromBuffer();
        semSignal(s);
        consume ();
    }
}
```

Which one of the following is TRUE?

- (a) The producer will be able to add an item to the buffer, but the consumer can never consume it.
- (b) The consumer will remove no more than one item from the buffer.
- (c) Deadlock occurs if the consumer succeeds in acquiring semaphore s when the buffer is empty.
- (d) The starting value for the semaphore n must be 1 and not 0 for deadlock-free operation.

14. [MCQ] [GATE-2013 : 2M]

A certain computation generates two arrays a and b such that $a[i] = f(i)$ for $0 \leq i < n$ and $b[i] = g(a[i])$ for $0 \leq i < n$. Suppose this computation is decomposed into two concurrent processes X and Y such that X computes the array a and Y computes the array b . The processes employ two binary semaphores R and S , both initialized to zero. The array a is shared by the two processes. The structure of the processes are shown below.

Process X

```
private i;
for (i = 0; i < n; i++)
{
```

Process Y

```
private i;
for(i = 0; i < n; i++)
{
```

```

a[i] = f(i);      EntryY(R, S);
ExitsX(R, S);    b[i] = g(a[i]);
}                }

```

Which one of the following represents the CORRECT implementations of ExitsX and EntryY?

(a)

```

Exits X (R, S)
{
    P(R);
    V(S);
}
Entry Y (R, S) {
    P(S);
    V(R);
}

```

(b)

```

Exits X (R, S)
{
    V(R);
    V(S);
}
Entry Y (R, S) {
    P(R);
    P(S);
}

```

(c)

```

Exits X (R, S)
{
    P(S);
    V(R);
}
Entry Y (R, S){
    V(S);
    P(R);
}

```

(d)

```

Exits X (R, S)
{
    V(R);
    P(S);
}
Entry Y (R, S){

```

```

V(S);
P(R);
}

```

15. [MCQ]**[GATE-2013 : 2M]**

A shared variable x , initialized to zero, is operated on by four concurrent process W, X, Y, Z as follows. Each of the processes W and X reads x from memory, increments by one, stores it to memory, and then terminates. Each of the processes Y and Z reads x from memory, decrements by one, stores it to memory and then terminates. Each process before reading x invokes the P operations (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What is the maximum possible value of x after all processes complete execution?

- (a) -2 (b) -1
(c) 1 (d) 2

16. [MCQ]**[GATE-2013 : 1M]**

Three concurrent processes X, Y and Z execute three different code segments that access and update certain shared variables. Process X execute the P operation (i.e., wait) on semaphores a, b and c ; process Y executes the P operation on semaphores b, c and d ; process Z executes the P operation on semaphores c, d and a before entering the respective code segments. After completing the execution of its code segments each process invokes the V operation (i.e., signal) on its three semaphores. All semaphores are binary semaphores initialized to one. Which one of the following represents a deadlock-free order of invoking the P operations by the processes?

- (a) $X: P(a)P(b)P(c)$ (b) $X: P(b)P(a)P(c)$
 $Y: P(b)P(c)P(d)$ $Y: P(b)P(c)P(d)$
 $Z: P(c)P(d)P(a)$ $Z: P(a)P(c)P(d)$
 (c) $X: P(b)P(a)P(c)$ (d) $X: P(a)P(b)P(c)$
 $Y: P(c)P(b)P(d)$ $Y: P(c)P(b)P(d)$
 $Z: P(a)P(c)P(d)$ $Z: P(a)P(c)P(d)$

17. [MCQ] [GATE-2010 : 2M]

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0 = 1$, $S_1 = 0$, $S_2 = 0$.

Process P0	Process P1	Process P2
<pre>while (true) { wait (S0); print '0'; release (S1); release (S2); }</pre>	<pre>wait (S1); release(S0);</pre>	<pre>wait (S2) release (S0);</pre>

How many times will process P0 print '0'?

- (a) At least twice (b) Exactly twice
(c) Exactly thrice (d) Exactly once

18. [MCQ] [GATE-2008 : 2M]

The P and V operations on counting semaphores, where s is a counting semaphore, are defined as follows:

$P(s): s = s - 1;$

if $s < 0$ then wait;

$V(s): s = s + 1;$

if $s \leq 0$ then wakeup a process waiting on s ;

Assume that P_b and V_b , the wait and signal operations on binary semaphores are provided. Two binary semaphores X_b and Y_b are used to implement the semaphore operations $P(s)$ and $V(s)$ as follows:

$P(s): P_b(X_b);$

$s = s - 1;$

if $(s < 0)$ {

$V_b(X_b);$

$P_b(Y_b);$

}

else $V_b(X_b);$

$V(s): P_b(X_b);$

$s = s + 1;$

if $(s \leq 0) V_b(Y_b);$

$V_b(X_b);$

The initial values of X_b and Y_b are respectively

- (a) 0 and 0 (b) 0 and 1
(c) 1 and 0 (d) 1 and 1

19. [MCQ] [GATE-2008 : 2]

The following is a code with two threads, producer and consumer, that can run in parallel. Further S and Q are binary semaphores equipped with the standard P and V operations.

Semaphore $S = 1$, $Q = 0$;

integer x ;

producer:

while (true) do

$P(S);$

$x = \text{produce}();$

$V(Q);$

done

consumer:

while (true) do

$P(Q);$

consume $(x);$

$V(S);$

done

Which of the following is TRUE about the program above?

- (a) The process can deadlock
(b) One of the threads can starve
(c) Some of the items produced by the producer may be lost
(d) Values generated and stored in ' x ' by the producer will always be consumed before the producer can generate a new value.

20. [MCQ] [GATE-2023-CS: 2M]

Consider the two functions incr and decr shown below.

There are 5 threads each invoking incr once, and 3 threads each invoking decr once, on the same shared variable X . The initial value of X is 10. Suppose there are two implementations of the semaphore s , as follows:

$\text{incr}\{$

$\text{wait}(s);$

$X = X + 1;$

$\text{Signal}(s);$

$\}$

$\text{decr}\{$

$\text{wait}(s);$

$X = X - 1;$

$\text{Signal}(s);$

$\}$

I-1: s is a binary semaphore initialized to 1.

I-2: s is a counting semaphore initialized to 2.

Let V_1 , V_2 be the values of X at the end of execution of all the threads with implementations I-1, I-2, respectively.

Which one of the following corresponds to the minimum possible values of V_1 , V_2 , respectively?

- (a) 15, 7 (b) 7, 7
(c) 12, 7 (d) 12, 8

21. [MCQ] [GATE-2022-CS: 1M]

Consider the following threads, T_1 , T_2 , and T_3 executing on a single processor, synchronized using three binary semaphore variables, S_1 , S_2 , and S_3 , operated upon using standard `wait()` and `signal()`. The threads can be context switched in any order and at any time.

T_1	T_2	T_3
<pre>while (true) { wait (S₃); print("C"); signal (S₂); }</pre>	<pre>while (true) { wait (S₁); print("B"); signal (S₃); }</pre>	<pre>while (true) {wait (S₂); print("A"); signal (S₁); }</pre>

Which initialization of the semaphores would print the sequence BCABCABCA.....?

- (a) $S_1 = 1; S_2 = 1; S_3 = 1$
- (b) $S_1 = 1; S_2 = 1; S_3 = 0$
- (c) $S_1 = 1; S_2 = 0; S_3 = 0$
- (d) $S_1 = 0; S_2 = 1; S_3 = 1$

Classical IPC Problems
22. [MCQ] [GATE-2015 : 1M]

Consider the following solution to the producer-consumer synchronization problem. The shared buffer size is N . Three semaphores empty, full and mutex are defined with respective initial value of 0, N and 1. Semaphore empty denotes the number of available slots in the buffer, for the consumer to read from. Semaphore full denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by P , Q , R and S , in the code below can be assigned either empty or full. The valid semaphore operations are: `wait ()` and `signal ()`.

Producer:	Consumer:
<pre>do { wait (P); wait (mutex); //Add item to buffer signal (mutex); signal (Q); }while (1);</pre>	<pre>do { wait (R); wait (mutex); //consume item from buffer signal (mutex); signal (S); }while (1);</pre>

Which one of the following assignment to P , Q , R and S will yield the correct solution?

- (a) P : full, Q : full, R : empty, S : empty
- (b) P : empty, Q : empty, R : full, S : full
- (c) P : full, Q : empty, R : empty, S : full
- (d) P : empty, Q : full, R : full, S : empty

Concurrency Mechanism
23. [NAT] [GATE-2019 : 1M]

Consider three concurrent process P_1 , P_2 and P_3 as shown below, which access a shared variable D that has been initialized to 100.

P_1	P_2	P_3
:	:	:
:	:	:
$D = D + 20$	$D = D - 50$	$D = D + 10$
:	:	:
:	:	:

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of $Y - X$ is _____.

24. [NAT] [GATE-2015 : 1M]

The following two functions P_1 and P_2 that share a variable B with an initial value of 2 execute concurrently.

$P_1()$	$P_2()$
<pre>{ C = B - 1; B = 2 * C }</pre>	<pre>{ D = 2 * B; B = D - 1; }</pre>

The number of distinct values that B can possibly take after the execution is _____.

ANSWER KEY

- | | | | |
|--------------|---------------|----------------|--------------|
| 1. (a) | 2. (c) | 3. (a) | 4. (b) |
| 5. (a) | 6. (a, d) | 7. (d) | 8. (a) |
| 9. (a, b, c) | 10. (a, b, d) | 11. (a) | 12. (7 to 7) |
| 13. (c) | 14. (c) | 15. (d) | 16. (b) |
| 17. (a) | 18. (c) | 19. (d) | 20. (c) |
| 21. (c) | 22. (c) | 23. (80 to 80) | 24. (3 to 3) |

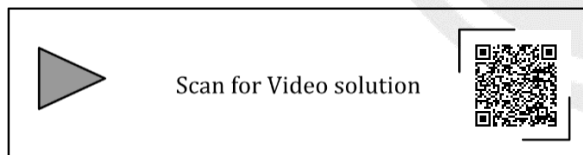
SOLUTIONS

1. (a)

The given algorithm is an updated version of Bakery Algorithm, this algorithm is basically for multi process solution.

The condition while $(t[j] \neq 0 \ \&\& \ t[j] \leq t[i])$; ensures that no two processes can be in critical section at the same time.

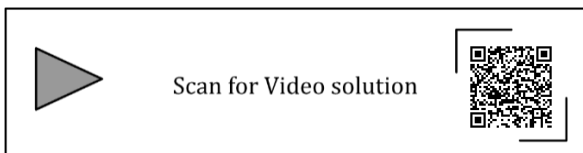
The given algorithm satisfies mutual exclusion but does not satisfies bounded waiting, progress, and deadlock. So, at most one process can be in the critical section at any time.



2. (c)

The given code is of strict alternation. Both processes are strictly changing the value of turn to 1 and 0. Strict alternation guarantees mutual exclusion but fails to guarantee process.

Therefore, This solution violates progress requirement. Hence, option (c) is correct.



3. (a)

Suppose

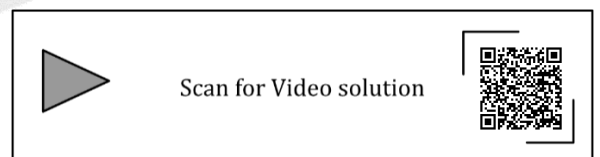
Process Y starts execution
made $\text{varQ} = \text{true}$; and pre-empted.

Process X starts execution
made $\text{varP} = \text{true}$;

Now both varP and varQ are true and both process X and Y can enter into CS simultaneously. Hence, no mutual exclusion.

Here deadlock is not possible, for deadlock the processes must stuck, here no process is stucked both are executing hence no deadlock.

Therefore, the proposed solution prevents deadlock but fails to guarantee mutual exclusion.



4. (b)

Fetch-And-Add(x, i), this will returns value of (x) and increment 'x' by 'i'.

Implementation :

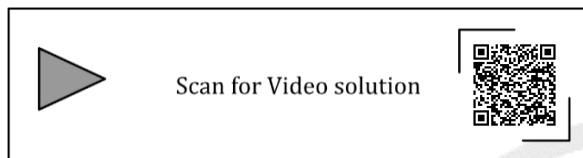
```
int fetch-And-Add (x, i)
{
    int rv;
    rv = x;
    x = x + i;
```

```

    return(rv);
}
while (F-A(L, 1))    // Entry section
<cs>
L = 0                // Exit section

```

A process acquires a lock only when L is equals to 0, when $L=1$, then the process repeats the while loop. There is no overflow because after every time l is incremented, L is again made equals to 1. Acquiring lock is successful only when Fetch_And_Add gets executed with $L = 0$.



5. (a)

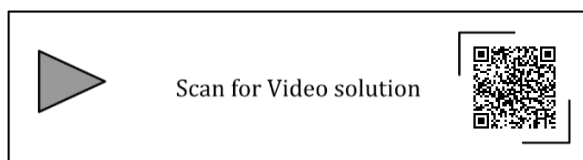
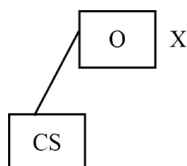
The given solution is deadlock free. So, statement I is true.

The given solution does not guarantee bounded waiting, this can lead to starvation. So, statement II is false.

There is no restriction on processes to enter the CS in a certain manner. Therefore, statement III is false.

The given solution guarantees mutual exclusion. So, statement IV is also incorrect.

Therefore, option A is the correct answer.



6. (a, d)

Here value of x is not assigned, but in C programming all the global and static variables are initialised to 0. y is a local variable.

So,

1. Process P_1 :

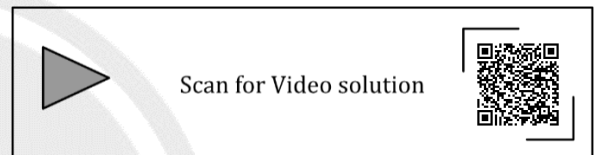
Two threads are created in main. Both threads execute `foo()` function and do not wait for each other and because of explicit locking mechanism [Acquire L1 and Release L1], mutual exclusion is there and no race condition will take place inside function `foo()`.

y is local variable, both threads will print its value as 1.

Because of wait in the main (); print (x) will only execute when both threads are completed. So, the x value will become 2.

2. Process P_2 :

P_2 will be executed same as P_1 . So, it will also print value of x as 2 and value of y as 1. Therefore, option A and D are true.



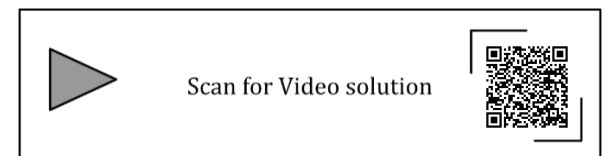
7. (d)

The question is asking for the minimum value of x and the minimum value of y together for which execution of P can result in a deadlock are:

So, checking for the minimum value $x = 1, y = 1$.

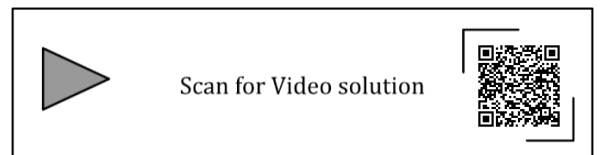
Deadlock is possible in this scenario the condition is process should perform down/wait operation twice. If down operation on lock is performed twice, then deadlock is possible.

Therefore, option D is correct.



8. (a)

The given code is purely Strict alternation method. Strict alternation always guarantee mutual exclusion but do not guarantee progress. So, option A is correct.



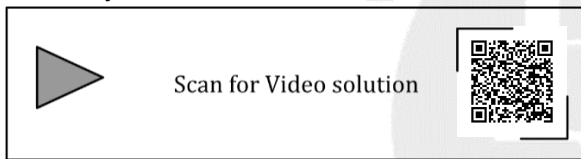
9. (a, b, c)

When every process is in the waiting state, a system is said to be in deadlock. This is comparable to a stop-and-go traffic situation.

When processes carry out repeated tasks without advancing the system (still producing no valuable work), the system is said to be in livelock. This is comparable to a traffic gridlock when some cars drive forward, strike a block, then reverse and move forward again.

Now, deadlock and livelock are mutually exclusive; in a system, only one of them can occur at any given time. However, they both suggest that the system will not advance, starving the associated processes in the process.

Now, regarding the query, any process can terminate another process and then obtain the necessary resource, and this can result in



10. (a, b, d)

The value of semaphore is initialised to 5.

If all the threads execute sequentially one by one, like first thread arrives and increases the value of counter by 1, then next thread increases it to 2 and so on. So, the final value will become 5. Therefore, option A is correct.

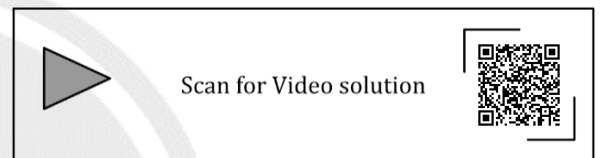
If the first thread arrives and performed counter++, increases the value of counter by 1, load the value in the register and gets pre-empted. Now subsequent thread arrives and completed the execution, increases the value of counter. At last, first thread will arrive (as it was pre-empted) to complete its execution it will store the value into register which is counter = 1 (the value loaded onto register before pre-emption). So, option B is also correct.

The value of counter cannot be 0, because counter's value has to be updated at least one time. Therefore,

it's maximum value will be 5 and minimum value will be 1. So, option C is incorrect.

Deadlock can be present in the system, if first thread arrives and performed step-5(wait(S);) and gets pre-empted, second thread arrives perform step-5 (wait(S);) and gets pre-empted and all other subsequent threads does the same. Now, the semaphore value will become 0 and if thread wants to perform step-6 (wait(S);), it will get blocked, and all the subsequent threads will be blocked after performing second wait(S); operation. None of the threads are able to execute further and hence a deadlock is possible involving all the threads. So, option D is also correct.

So, option A, B, and D are correct.



11. (a)

Given,

There are two semaphores a and b initialized to 1 and 0 respectively. Count is a shared variable it is initialized to 0.

1. CODE SECTION P
2. wait (a);
3. count = count + 1;
4. if (count == n) signal (b);
5. signal (a);
6. wait (b);
7. signal (b);
8. CODE SECTION Q

Process will arrive,

Complete the CODE SECTION P //line 1.

and then only it can perform wait (a); //line 2

now, it will increase the value of count to 1; //line 3

check if (count==n), if yes then perform signal (b); else we will not execute signal(b); //line 4

It will perform signal (a); //line 5

wait (b) [b was initialized to 0, so it will be blocked]. //line 6

Now, processes will continuously block till count $= n$. Once count becomes equals to n , it will execute `signal(b)` // line 4 and this unblocks one process, that unblocked process will execute `signal(b)` // line 7. Now, subsequent processes will be unblocked and can enter CODE SECTION Q //line 8.

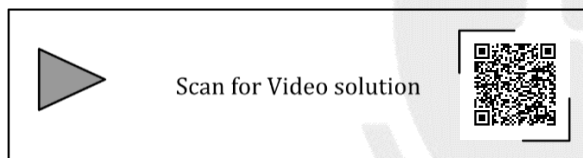
Means,

No processes can enter into CODE SECTION Q until N processes finishes CODE SECTION P and N processes has incremented the count variable.

OR

To execute CODE SECTION Q, N processes must completed their CODE SECTION P and has incremented the count variable.

So, It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P. Therefore, option A is correct.



12. (7)

Suppose, the initial value of $S = x$

Now, 20P means 20 down operations(-20) and 12 V means 12 up operations (+12).

It is saying that at least one operation should remain blocked.

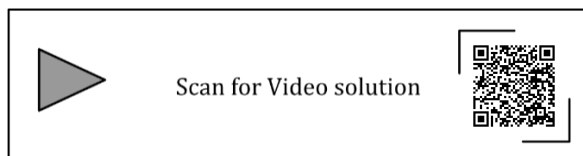
So, this can be written as,

$$x - 20 + 12 = -1$$

$$x - 8 = -1$$

$$x = 7$$

So, the largest initial value of S for which at least one operation will remain blocked is 7.



13. (c)

In the given code, if the producer will produce the item, then consumer will definitely consume it. So, the option A is incorrect.

If producer is producing two or more items then consumer can consume all of them, So, option B is incorrect.

If the buffer is empty, then

$$n = 0$$

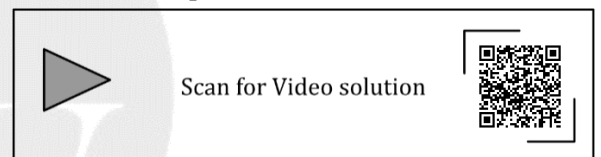
$$S \neq 0 \quad [\text{wait operation on semWait}(S)]$$

$\therefore n = 0$; consumer gets blocked

$S = 0$; producer gets blocked

Both the processes are blocked, therefore there exists a deadlock. Hence, option C is correct.

In option D, with $n = 1$, then also there will be a deadlock, So, option D is also incorrect.



14. (c)

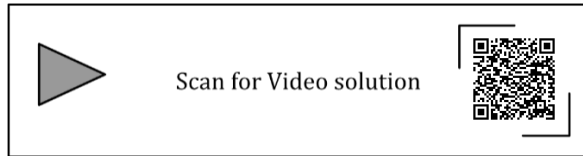
(a) With option A, deadlock happens.

(b) Process 'X' may compute all values (iterations) and process would be able to make only one iteration and then gets blocked. So, option B is also incorrect.

(c) Processes 'X' and 'Y' alternates and signal each other so that all iterations of both 'X' and 'Y' completes successfully

(d) Some of the signals issued by 'X' cannot be captured by 'Y'

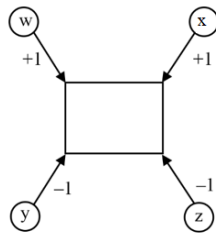
Therefore, process 'Y' will not be able to complete all iterations. So, option D is also incorrect.



15. (d)

The code can be written as:

```
CSEM S = 2;
Pri
i = W, X, Y, Z
{
    P(S);
    <update x>
    V(S);
}
```



W: Completes, $x = 1$

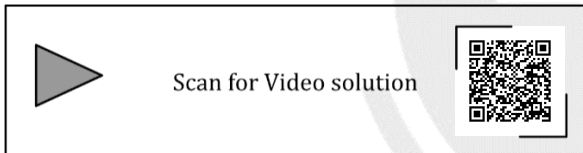
X: Load ; increment; Preempted, $x = 2$

Y: Completes

Z: Completes

X: Store, $x = 2$ is stored as the final value

Therefore, 2 is the correct answer hence option D is true.



16. (b)

In option A:

X: P(a) // $a = 0$

Y: P(b) // $b = 0$

Z: P(c) // $c = 0$

X: P(a)P(b) // $b = \text{blocked}$

Y: P(b)P(c) // $c = \text{blocked}$

Z: P(c)P(d) P(a) // $d = 0$, $a = \text{blocked}$

Here we can have deadlock.

In option B:

X: P(b) // $b = 0$

Y: P(b) // $b = \text{blocked}$

Z: P(a) // $a = 0$

X: P(b)P(a) // $a = \text{blocked}$

Y: P(b)P(c) // already blocked by b

Z: P(a)P(c) // $c = 0$

X: P(b)P(a)P(c) // already blocked by a

Y: P(b)P(c)P(d) // already blocked by b

Z: P(a)P(c)P(d) // $d = 0$ [Success]

No deadlock.

If one process is successful other processes can also complete successfully. So, option B is the correct answer.

In option C:

X: P(b) // $b = 0$

Y: P(c) // $c = 0$

Z: P(a) // $a = 0$

X: P(b)P(a)P(c) // $a = \text{blocked}$

Y: P(c)P(b)P(d) // $b = \text{blocked}$

Z: P(a)P(c)P(d) // $c = \text{blocked}$.

Here we can have deadlock.

In option D:

X: P(a) // $a = 0$

Y: P(c) // $c = 0$

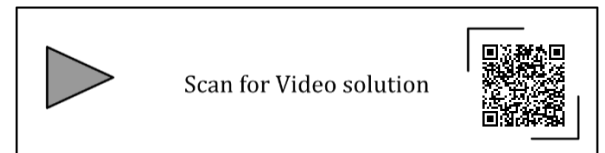
Z: P(a) // $a = \text{blocked}$

X: P(a) P(b)P(c) // $b = 0$

Y: P(c)P(b)P(d) // $b = \text{blocked}$

Z: P(a)P(c)P(d)

Here also deadlock is possible.



17. (a)

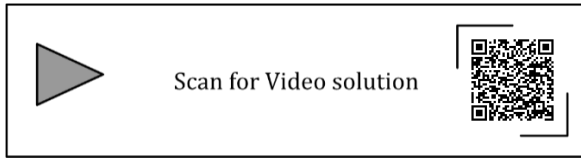
If the process run in following order

- $P_0; P_1; P_2; P_0; P_0 \rightarrow$ "0" will be printed 2 times

If the process run in following order

- $P_0; P_1; P_0; P_2; P_0; P_0 \rightarrow$ "0" will be printed 3 times

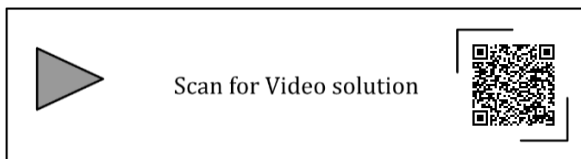
So, "0" will be printed maximum 3 times, and minimum 2 times. Therefore, option A is correct.



18. (c)

Initial

The value of X_b compulsarily should be 1, So that any one of $P_b(X_b)$ should be successful in the beginning itself.

 $Y_b = 0$


19. (d)

Only after the producer has made the thing may the consumer consume it, and only after the consumer has consumed the item can the producer manufacture it again (except for the first time).

Let's go over how this code functions.

The parallel execution of Producer and Consumer is mentioned.

Producer:

S value is 1 in st1, P(S) on S reduces it to 0, and st2 produces the x item.

Q value for st3 is 0. V(Q) on Q results in 1.

Since this is an endless while loop, it ought to loop endlessly. S is already 0 in the subsequent while loop iteration; further P(S) on 0 sends P to S's blocked list. Producer is thus obstructed.

Consumer:

P(Q) on Q causes Q to equal 0 and then eats the thing.

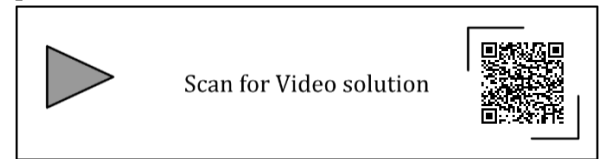
Instead of setting S to 1 with V(S) on S, the consumer now reawakens the stalled process on Q's queue. Process P thus becomes active. Since P was blocked at statement 1, it continues from statement 2. P then creates the subsequent item. Therefore, a product is consumed by a consumer before a producer creates a new one.

The right answer is (d).

Option for this query:

Deadlock cannot occur because both the producer and the consumer are using distinct semaphores (there is no hold and wait). Likewise, there is no starvation since there is a change in the relationship between the producer and the consumer, which also results in bounded waiting.

(C) The producer might lose some of the products it produces, but



20. (c)

Case I: Let's the minimum possible value of x for v1, when semaphore 's' is binary semaphore initialized to 1.

$P_4 \dots P_8 \Rightarrow \text{Incr}() \text{Decr}() \Rightarrow P_1, P_2, P_3$

- | | |
|----------------|------------------|
| 1. Wait (s); | 1. Wait(s); |
| 2. $x = x + 1$ | 2. $x = x - 1$; |
| 3. signal(s); | 3. Signal(s) |

Semaphore s = 1 0 1

$P_1 = \text{Decr}()$

1. wait (s); s = 0

2. Read x; x = 10

Now, if you pre-empt P_1 , before performing decrement operation then other process would not be able to execute on 'x' as semaphore value is s = 0.

$x - 1: 10 - 1: 9$

$x = 9$; update x by 9

3. signal(s); s = 1

$P_2 \text{ Decr}()$

1, 2, 3: Final value of x = 8

$P_3: \text{Decr}()$

1, 2, 3 : Final value of x = 7

$P_4 \ P_5: \text{Incr}()$

1,2,3 Increment x 5 times: $x \Rightarrow 12$

Hence, minimum possible value is 12 for V_1 .

Case II: Let's find the minimum possible value of x for V_2 . when semaphore S is counting semaphore

initialized to 2.

P_1 : Decr ()

1. wait (s): $s \neq 2$ 1

2. read (x); $x = 10$

Preempt Process P_1 .

Now. execute Incr()

$P_4 \dots P_5$: Incr()

1, 2, 3: Final x value will be, $x = 15$

Now. revoke the process P_1 from pre-emption

2. Perform $x - 1$: $10 - 1 = 9$

Update x: $x \equiv 9$

P_2 : Decr()

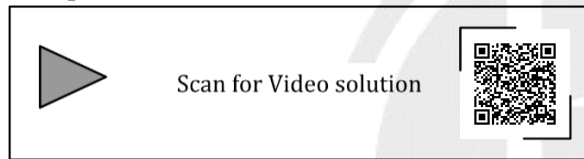
1,2,3 $x = 8$

P_3 Decr()

1,2,3 $x = 7$

Hence, find value of V_2 is 7 for counting semaphore

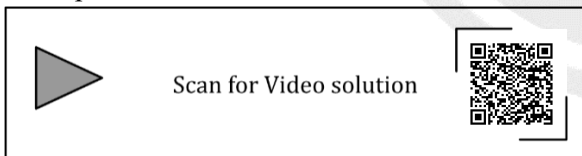
So, option C is the correct answer.



21. (c)

We need the output to be BCABCABCA, i.e. it should start with B, means T_2 should be 1 and T_1, T_3 should be zero. This means S_1 should be 1; S_2 and S_3 would be 0.

So, option C is the correct answer.



22. (c)

Given,

Empty is initialized with 0.

Full is initialized with N.

Mutex is initialized with 1.

Initially the buffer is empty therefore, empty is initialized to 0. And if buffer is full then Full will be 0 as there will be no slots available.

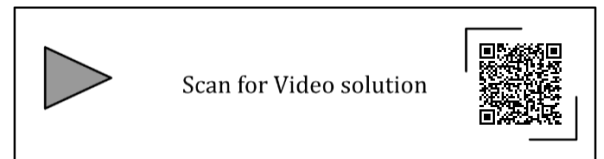
The given code is of classical IPC problem, producer consumer problem.

So, in Producer; wait (P), if P is full, then producer will be blocked. Means P should be full. So, option B and D are eliminated and are incorrect.

Now, if we are performing wait (P) // means wait on full

Then, signal(Q) operation should be on empty.

Hence, option A is also incorrect and option C is the correct answer.



23. (80)

Given,

$D = 100$

X is minimum value

Y is maximum value

To calculate maximum value:

P_1 : $D = D + 20 \quad \therefore D = 120$

P_3 : $D = D + 10$ // executed load, executed increment and got pre-empted. [Means the value of register is 130 here but this value is not stored into register yet.]

P_2 : $D = D - 50$ // value of D was 130 and now $130 - 50 = 80$, we will store 80 into register.

Now, P_3 will complete its execution and perform store operation. When P_3 preempted the value of register was 130, so it will store value as 130

Therefore, **Y = 130**

Similarly, To calculate minimum value:

P_2 : $D = D - 50$ // Load the value; Decrement the value of register [$100 - 50 = 50$]; Pre-empt

Execute P_1 ;

Execute P_3 ;

Complete execution of P_2 , perform store operation. The final value of register will be 50.

Therefore, **X = 50**

So, the difference between maximum and minimum value is $Y - X = 130 - 50 = 80$



Scan for Video solution



24. (3)

Initially $B = 2$

P1()

{

1. $C = B - 1$;2. $B = 2 * C$;

}

P2()

{

3. $D = 2 * B$;4. $B = D - 1$;

}

CASE I:

1. $C = B - 1 \Rightarrow 2 - 1 \Rightarrow 1$
2. $B = 2 * C \Rightarrow 2 * 1 \Rightarrow 2$
3. $D = 2 * B \Rightarrow 2 * 2 \Rightarrow 4$
4. $B = D - 1 \Rightarrow 4 - 1 \Rightarrow 3$

 $B = 3$ **CASE II:**

3. $D = 2 * B \Rightarrow 2 * 2 \Rightarrow 4$
4. $B = D - 1 \Rightarrow 4 - 1 \Rightarrow 3$
1. $C = B - 1 \Rightarrow 3 - 1 \Rightarrow 2$
2. $B = 2 * C \Rightarrow 2 * 2 \Rightarrow 4$

 $B = 4$ **CASE III:**

1. $C = B - 1 \Rightarrow 2 - 1 \Rightarrow 1$
3. $D = 2 * B \Rightarrow 2 * 2 \Rightarrow 4$
4. $B = D - 1 \Rightarrow 4 - 1 \Rightarrow 3$
2. $B = 2 * C \Rightarrow 2 * 1 \Rightarrow 2$

 $B = 2$ **CASE IV:**

3. $D = 2 * B \Rightarrow 2 * 2 \Rightarrow 4$
1. $C = B - 1 \Rightarrow 2 - 1 \Rightarrow 1$
4. $B = D - 1 \Rightarrow 4 - 1 \Rightarrow 3$
2. $B = 2 * C \Rightarrow 2 * 1 \Rightarrow 2$

 $B = 2$

The number of distinct values that B can possibly take after the execution is 3 [3, 4, 2].



Scan for Video solution



CHAPTER

3

DEADLOCKS

Deadlock Handling

1. [MCQ] [GATE-2019 : 2M]

Consider the following snapshot of a system running n concurrent processes. Process i is holding X_i instances of a resources R , $1 \leq i \leq n$. Assume that all instances of R are currently in use. Further, for all i , process i can place a request for at most Y_i additional instances of R while holding the X_i instances it already has. Of the n processes, there are exactly two processes p and q such that $Y_p = Y_q = 0$. Which one of the following conditions guarantees that no other process apart from p and q can complete execution?

- $X_p + X_q < \text{Min} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $X_p + X_q < \text{Max} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $\text{Min}(X_p, X_q) \geq \text{Min} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $\text{Min}(X_p, X_q) \leq \text{Max} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$

2. [MCQ] [GATE-2018 : 2M]

In a system, there are three types of resources: E, F and G. Four processes P_0, P_1, P_2 and P_3 execute concurrently. At the outset, the processes have declared their maximum resource requirements using a matrix named Max as given below. For example, $\text{Max}[P_2, F]$ is the maximum number of instances of F that P_2 would require. The number of instances of the resources allocated to the various processes at any given state is given by a matrix named Allocation.

Consider a state of the system with the Allocation matrix as shown below and in which 3 instances of E and 3 instances of F are the only resources available.

Form the perspective of deadlock avoidance, which one of the following is true?

Allocation			
	E	F	G
P_0	1	0	1
P_1	1	1	2
P_2	1	0	3
P_3	2	0	0

Max			
	E	F	G
P_0	4	3	1
P_1	2	1	4
P_2	1	3	3
P_3	5	4	1

- The system is in safe state.
- The system is not in safe state, but would be safe if one more instance of E were available.
- The system is not in safe state, but would be safe if one more instance of F were available.
- The system is not in safe state, but would be safe if one more instance of G were available.

3. [MCQ] [GATE-2017 : 2M]

A multithreaded program P executes with x number of threads and uses y number of locks for ensuring mutual exclusion while operating on shared memory locations. All locks in the program are non-reentrant, i.e., if a thread holds a lock l , then it cannot re-acquire lock l without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The minimum value of x and the minimum value of y together for which execution of P can result in a deadlock are:

- $x = 1, y = 2$
- $x = 2, y = 1$
- $x = 2, y = 2$
- $x = 1, y = 1$

4. [MCQ] [GATE-2017 : 2M]

A system shares 9 tape drives. The current allocation and maximum requirement of tape drives for that processes are shown below:

Process	Current Allocation	Maximum Requirement
P1	3	7
P2	1	6
P3	3	5

Which of the following best describes current state of the system?

- (a) Safe, Deadlocked
- (b) Safe, Not Deadlocked
- (c) Not Safe, Deadlocked
- (d) Not Safe, Not Deadlocked

5. [MCQ] [GATE-2015: 2M]

Consider the following policies for preventing deadlock in a system with mutually exclusive resources.

- Processes should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released.
- The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers.
- The resources are numbered uniquely, and processes are allowed to request for resources only in decreasing resource numbers.
- The resources are numbered uniquely. A process is allowed to request only for a resource with resource number larger than its currently held resources.

Which of the above policies can be used for preventing deadlock?

- (a) Any one of I and III but not II or IV
- (b) Any one of I, III and IV but not II
- (c) Any one of II and III but not I or IV
- (d) Any one of I, II, III and IV

6. [MCQ] [GATE-2014 : 2M]

An operating system uses the Banker's algorithm for deadlock avoidance when managing the allocation of three resource types X, Y and Z to three processes P₀, P₁, and P₂. The table given below represents the current system state. Here, the Allocation matrix

shows the current number of resources of each type allocated to each process and the Max matrix shows the maximum number of resources of each type required by each process during its execution.

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

There are 3 units of type X, 2 units of type Y and 2 units of type Z still available. The system is currently in a safe state. Consider the following independent requests for additional resources in the current state:

REQ 1: P₀ requests 0 units of X, 0 units of Y and 2 units of Z

REQ 2: P₁ requests 2 units of X, 0 units of Y and 0 units of Z

Which one of the following is TRUE?

- (a) Only REQ 1 can be permitted.
- (b) Only REQ 2 can be permitted.
- (c) Both REQ 1 and REQ 2 can be permitted
- (d) Neither REQ 1 nor REQ 2 can be permitted

7. [MCQ] [GATE-2008 : 2M]

Which of the following is NOT true of deadlock prevention and deadlock avoidance schemes?

- (a) In deadlock prevention, the request for resources is always granted if the resulting state is safe
- (b) In deadlock avoidance, the request for resources is always granted if the resulting state is safe
- (c) Deadlock avoidance is less restrictive than deadlock prevention
- (d) Deadlock avoidance requires knowledge of resource requirements a priori.

Resource Allocation Graph

8. [MSQ] [GATE-2022: 1M]

Which of the following statements is/are TRUE with respect to deadlocks?

- (a) Circular wait is a necessary condition for the formation of deadlock.
- (b) In a system where each resource has more than one instance, a cycle in its wait-for graph indicates the presence of a deadlock.

- (c) If the current allocation of resources to processes leads the system to unsafe state, then deadlock will necessarily occur.
- (d) In the resource-allocation graph of a system, if every edge is an assignment edge, then the system is not in deadlock state.

9. [NAT] [GATE-2018 : 1M]

Consider a system with 3 processes that share 4 instances of the same resource type. Each process can request a maximum of K instances. Resource instances can be requested and released only one at a time. The largest value of K that will always avoid deadlock is _____.

10. [MCQ] [GATE-2015 : 2M]

A system has 6 identical resources and N processes competing for them. Each process can request at most 2 resources. Which one of the following values of N could lead to a deadlock?

- (a) 1 (b) 2
(c) 3 (d) 4

11. [NAT] [GATE-2014: 2M]

A system contains three programs, and each requires three tape units for its operation. The minimum number of tape units which the system must have such that deadlocks never arise is _____.

12. [MCQ] [GATE-2010 : 2M]

A system has n resources $R_0, R_1, \dots, R_{n-1}$, and k processes $P_0, P_1, \dots, P_{k-1}$. The implementation of the resource request logic of each process P_i is as follows:

```
if (i % 2 == 0) {
    if (i < n) request  $R_i$ ;
    if (i + 2 < n) request  $R_{i+2}$ ;
}
else {
    if (i < n) request  $R_{n-i}$ ;
    if (i + 2 < n) request  $R_{n-i-2}$ ;
}
```

In which one of the following situations is a deadlock possible?

- (a) $n = 40, k = 26$ (b) $n = 21, k = 12$
(c) $n = 20, k = 10$ (d) $n = 41, k = 19$

13. [MCQ] [GATE-2009 : 2M]

Consider a system with 4 types of resources R_1 (3 units), R_2 (2 units), R_3 (3 units), R_4 (2 units). A non-preemptive resource allocation policy is used. At any given instance, a request is not entertained if it cannot be completely satisfied. Three processes P_1, P_2, P_3 request the resources as follows if executed independently.

Process P_1 :	Process P_2 :	Process P_3 :
t = 0 : requests 2 units of R_2	t = 0: requests 2 units of R_3	t = 0: requests 1 unit of R_4
t = 1: requests 1 unit of R_3	t = 2 : requests 1 unit of R_4	t = 2: requests 2 units of R_1
t = 3: requests 2 units of R_1	t = 4 : requests 1 unit of R_1	t = 5: releases 2 units of R_1
t = 5: releases 1 unit of R_2 and 1 unit of R_1	t = 6 : releases 1 unit of R_3	t = 7: requests 1 unit of R_2
t = 7 : releases 1 unit of R_3	t = 8 : Finishes	t = 8: requests 1 unit of R_3
t = 8 : requests 2 units of R_4		t = 9: Finishes
t = 10 : Finishes		

Which one of the following statements is TRUE if all three processes run concurrently starting at time $t = 0$?

- (a) All processes will finish without any deadlock.
(b) Only P_1 and P_2 will be in deadlock.
(c) Only P_1 and P_3 will be in deadlock.
(d) All three processes will be in deadlock

14. [MCQ] [GATE-2008: 2M]

An operating system implements a policy that requires a process to release all resources before making a request for another resource.

Select the TRUE statement from the following:

- (a) Both starvation and deadlock can occur.
(b) Starvation can occur but deadlock cannot occur.
(c) Starvation cannot occur but deadlock can occur.
(d) Neither starvation nor deadlock can occur.



ANSWER KEY

- | | | | |
|-------------|---------|--------------|-----------|
| 1. (a) | 2. (a) | 3. (d) | 4. (b) |
| 5. (d) | 6. (b) | 7. (a) | 8. (a, d) |
| 9. (2 to 2) | 10. (d) | 11. (7 to 7) | 12. (b) |
| 13. (a) | 14. (b) | | |

SOLUTIONS

1. (a)

p_i	alloc	request	Avail/R
p_1	x_1	y_1	0
p_2	x_2	y_2	
.	.	.	
.	.	.	
p_p	x_p	0	
$P=$	x_p	0	
.	.	.	
.	.	.	
p_n	x_n	y_n	

 $< P_p, p_q$ 't' : $(x_p + x_q)$: Avail $(x_p + x_q) < \min(y_k)$ $k \neq (p, q)$ 

Scan for Video solution



P1	2 1 4	1 1 2	1 0 2	
P2	1 3 3	1 0 3	0 3 0	
P3	5 4 1	2 0 0	3 4 1	

P0 can be executed. And the available will be

Pid	Max E F G	Allocation E F G	Need E F G	Available E F G
Po	4 3 1	1 0 1	3 3 0	3 0 0
P1	2 1 4	1 1 2	1 0 2	4 3 1
P2	1 3 3	1 0 3	0 3 0	
P3	5 4 1	2 0 0	3 4 1	

Now, P2's can be satisfied, so P2 will be executed and

Pid	Max E F G	Allocation E F G	Need E F G	Available E F G
Po	4 3 1	1 0 1	3 3 0	3 0 0
P1	2 1 4	1 1 2	1 0 2	4 3 1
P2	1 3 3	1 0 3	0 3 0	5 3 4
P3	5 4 1	2 0 0	3 4 1	

Now, P1's need can be satisfied, so P1 will be executed and

2. (a)

Resource Allocation Matrix:

Pid	Max E F G	Allocation E F G	Need E F G	Available E F G
Po	4 3 1	1 0 1	3 3 0	3 0 0

Pid	Max E F G	Allocation E F G	Need E F G	Available E F G
P ₀	4 3 1	1 0 1	3 3 0	3 0 0
P ₁	2 1 4	1 1 2	1 0 2	4 3 1
P ₂	1 3 3	1 0 3	0 3 0	5 3 4
P ₃	5 4 1	2 0 0	3 4 1	6 4 6

Now, we can satisfy needs of P₃. Therefore,

Pid	Max E F G	Allocation E F G	Need E F G	Available E F G
P ₀	4 3 1	1 0 1	3 3 0	3 0 0
P ₁	2 1 4	1 1 2	1 0 2	4 3 1
P ₂	1 3 3	1 0 3	0 3 0	5 3 4
P ₃	5 4 1	2 0 0	3 4 1	6 4 6
				8 4 6

Safe sequence will be: $\langle P_0, P_2, P_1, P_3 \rangle$

System state is safe

So, option A is True.



4. (b)

Resource Allocation Matrix:

Pid	Max R	Allocation R	Need R	Avail R
P ₁	7	3	4	2
P ₂	6	1	5	
P ₃	5	3	2	

P₃'s need can be satisfied and the available will become

Pid	Max R	Allocation R	Need R	Avail R
P ₁	7	3	4	2
P ₂	6	1	5	5
P ₃	5	3	2	

Now, P₂'s need can be fulfilled.

Pid	Max R	Allocation R	Need R	Avail R
P ₁	7	3	4	2
P ₂	6	1	5	5
P ₃	5	3	2	6

Now, P₁'s need can be fulfilled.

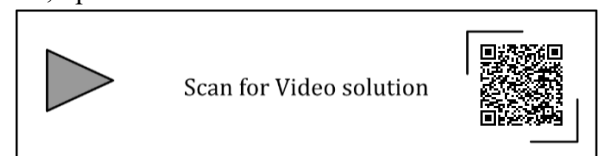
Pid	Max R	Allocation R	Need R	Avail R
P ₁	7	3	4	2
P ₂	6	1	5	5
P ₃	5	3	2	6
				9

R = 9

Safe sequence: P₃, P₂, P₁

∴ System is safe and not deadlocked

So, option B is correct.



5. (d)

A deadlock will never take place if any one of the following condition is preempted.

1. Hold & Wait
2. Circular Wait

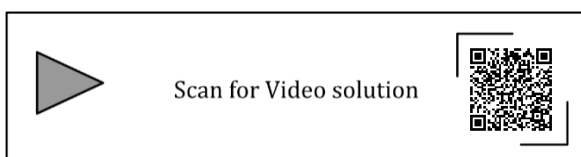
3. (d)

The question is asking for the minimum value of x and the minimum value of y together for which execution of P can result in a deadlock are:

So, checking for the minimum value $x = 1, y = 1$.

Deadlock is possible in this scenario the condition is process should perform down/wait operation twice. If down operation on lock is performed twice, then deadlock is possible.

Therefore, option D is correct.



3. Mutual Exclusion

4. No-Preemption

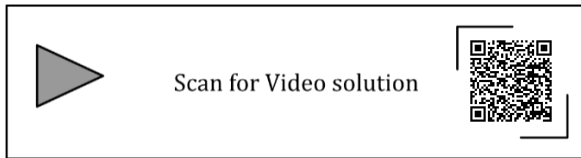
Statement I violates hold & wait.

Statement II violates circular wait.

Statement III violates circular wait.

Statement IV also violates circular wait by making dependency graph acyclic.

Therefore, option D is correct.



6. (b)

Resource allocation table:

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 1	8 4 2	3 2 2
P ₁	6 2 0	3 2 0	3 0 0	
P ₂	3 3 3	2 1 1	1 2 2	

Request 1 needs [0, 0, 2] resources for process P₁, then the system will become

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 3	8 4 0	3 2 0
P ₁	6 2 0	3 2 0	3 0 0	
P ₂	3 3 3	2 1 1	1 2 2	

Now, we can satisfy need of process P₁, after allotting resources to P₁,

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 3	8 4 0	3 2 0
P ₁	6 2 0	3 2 0	3 0 0	6 4 0
P ₂	3 3 3	2 1 1	1 2 2	

Now, with the given available resources, the need of neither P₂ nor P₀ can be satisfied, system is unsafe, Therefore, Req₁ cannot be granted.

Now, checking for Req₂. Request 2 needs [2, 0, 0]

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 1	8 4 2	1 2 2
P ₁	6 2 0	5 2 0	1 0 0	
P ₂	3 3 3	2 1 1	1 2 2	

P₁'s need can be satisfied and available will become

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 1	8 4 2	1 2 2
P ₁	6 2 0	5 2 0	1 0 0	6 4 2
P ₂	3 3 3	2 1 1	1 2 2	

Now, P₂'s request can be granted and after completing P₂ the new available will become.

Pid	Max X Y Z	Alloc X Y Z	Need X Y Z	Avail X Y Z
P ₀	8 4 3	0 0 1	8 4 2	1 2 2
P ₁	6 2 0	5 2 0	1 0 0	6 4 2
P ₂	3 3 3	2 1 1	1 2 2	8 5 3

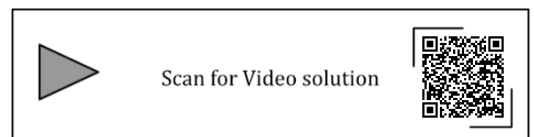
With 8, 5, 3. Need of P₀ can be fulfilled easily.

Hence Req₂ can be granted.

(a) System is unsafe
∴ Req₁ is Not granted

(b) < P₁, P₂, P₀>

System is safe
Req₂ is granted



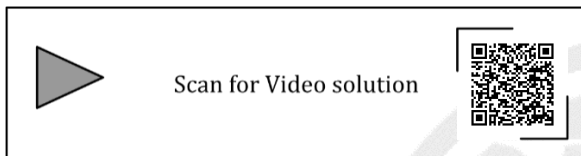
7. (a)

In deadlock prevention, we need to ensure one of the four conditions (hold & wait, circular wait, mutual exclusion and no-pre-emption) to avoid deadlock, but there may be a case where resulting state is safe but the resource request is rejected. So, option A is not always true.

In deadlock avoidance, if the resulting state is safe, then request is always granted and deadlock avoidance is less restrictive than deadlock prevention. So, option B, C both are true.

In deadlock avoidance, knowledge of resource requirements must be known apriori. True.

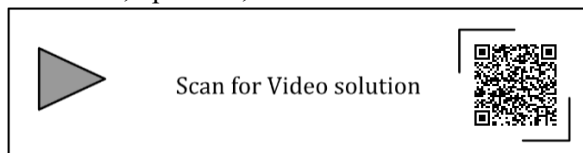
Therefore, option A is correct.



8. (a, d)

- (a) Circular wait is a necessary condition for the formation of deadlock. TRUE
- (b) In a system where each resource has more than one instance, a cycle in its wait-for graph indicates the presence of a deadlock. FALSE. Every cycle is not a deadlock but every deadlock have a cycle.
- (c) If the current allocation of resources to processes leads the system to unsafe state, then deadlock will necessarily occur. FALSE. Every unsafe state is not deadlock.
- (d) In the resource-allocation graph of a system, if every edge is an assignment edge, then the system is not in deadlock state. TRUE. Only hold without wait doesn't cause deadlock.

Therefore, option A, D are correct.



9. (2 to 2)

Given,

$$R = 4$$

To avoid deadlock, so each process can have maximum of $k-1$ resources.

Required	
P_1	$K-1$
P_2	$K-1$
P_3	$K-1$

$$R = 4$$

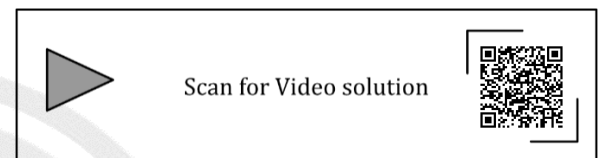
$$\text{Condition for deadlock} = (3K - 3) + 1$$

$$\Rightarrow 3K - 2 = 4$$

$$\Rightarrow 3K = 6$$

$$\therefore K = 2$$

So, 2 is the correct answer.

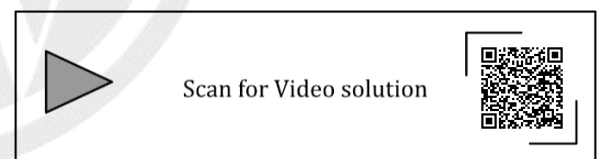


10. (d)

$$P_i = 2 (R)$$

If we allot 2 resources to 3 processes, then total resource required = 6. Hence, no deadlock.

The number of processes, so the system should be in deadlock will be 4. Each processes requires 2 resources at most. Total resource required = 8.



11. (7 to 7)

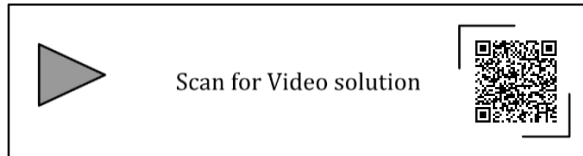
Number of processes, $N = 3$

Min (R)

$$P_i \rightarrow 3 (R)$$

Minimum number of resource unit so the deadlock never arises is, 7, upto 6 resources each process can have 2 resources each and result into deadlock. But with 7 resources, at least one processes' need will be satisfied and hence other processes can also proceed.

$$\begin{array}{l} P_1 - 2 \\ P_2 - 2 + 1 = 7 \\ P_3 - 2 \end{array}$$



12. (b)

- Based on Resource Request logic, Even Numbered processes are Requesting Even number resource.
- For Preventing/Avoiding Deadlock, odd numbered processes should request odd numbered resources.
- In the else part, when 'n' is odd, then process will request even numbered resources & may form cycle, which will lead to Deadlock.

∴ Only in option (B) & (D) there is a possibility of deadlock.

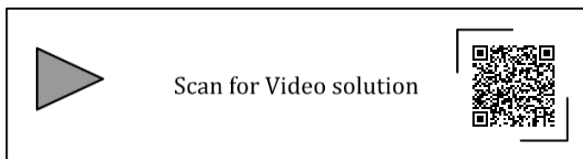
Option (B):

$$M = 2k; k = 12$$

$P_0 \rightarrow R_0, R_2$	$P_1 \rightarrow R_{20}, R_{18}$
$P_2 \rightarrow R_2, R_4$	$P_3 \rightarrow R_{18}, R_{16}$
$P_4 \rightarrow R_4, R_6$	$P_5 \rightarrow R_{16}, R_{14}$
$P_6 \rightarrow R_6, R_8$	$P_7 \rightarrow R_{14}, R_{12}$
$P_8 \rightarrow R_8, R_{10}$	$P_9 \rightarrow R_{12}, R_{10}$
$P_{10} \rightarrow R_{10}, R_{11}$	$P_{11} \rightarrow R_{10}, R_8$

$$\begin{array}{l} P_8 \rightarrow R_8, R_{10} \\ P_{11} \rightarrow R_{10}, R_8 \end{array} \text{ Deadlock}$$

Similarly, for Option (d) there is no – overlap with resources.

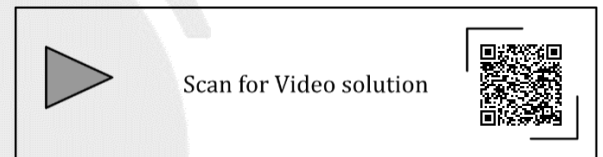


13. (a)

List of events at time T_i

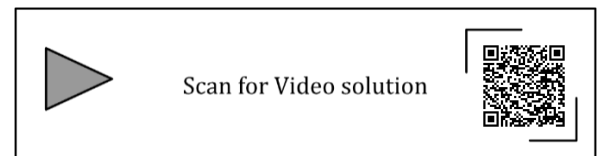
$T_0 : P_1 \rightarrow 2(R_2); P_2 \rightarrow 2(R_3); P_3 \rightarrow 1(R_4)$
$T_1 : P_1 \rightarrow 1(R_3);$
$T_2 : P_2 \rightarrow 1(R_4); P_3 \rightarrow 2(R_1);$
$T_3 : P_1 \rightarrow 2(R_1);$
$T_4 : P_2 \rightarrow 1(R_1)$
$T_5 : P_1 \rightarrow \text{releases 1 unit of } R_2; 1 \text{ unit of } R_1; P_3 \text{ is releasing 2 } (R_1)$
$T_6 : P_2 \rightarrow \text{releases 1 unit of } R_3$
$T_7 : \text{event is satisfied}$
$T_8 : \text{event is satisfied}$
$T_9 : \text{event is satisfied}$
$T_{10} : \text{event is satisfied}$

No Deadlock



14. (b)

In such a policy starvation can occur as every time a process requests for a resource it has to release all the acquired resources. If the process has not utilized acquired resources properly then it has to repeat this when the process requests for another resource and improper utilization of resources can lead to starvation.



CHAPTER

4

MEMORY MANAGEMENT

Memory Management Techniques

1. [MCQ] [GATE-2020: 2M]

Consider allocation of memory to a new process. Assume that none of the existing holes in the memory will exactly fit the process's memory requirement. Hence, a new hole of smaller size will be created if allocation is made in any of the existing holes. Which one of the following statements is TRUE?

- (a) The hole created by next fit is never larger than the hole created by best fit.
- (b) The hole created by first fit is always larger than the hole created by next fit.
- (c) The hole created by best fit is never larger than the hole created by first fit.
- (d) The hole created by worst fit is always larger than the hole created by first fit.

2. [MCQ] [GATE-2015 : 2M]

Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB, and 250 KB, where KB refers to kilobyte. These partitions need to be allotted to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order. If the best fit algorithm is used, which partitions are NOT allotted to any process?

- (a) 200 KB and 300 KB (b) 200 KB and 250 KB
- (c) 250 KB and 300 KB (d) 300 KB and 400 KB

Paging

3. [NAT] [GATE-2023 : 2M]

Consider a computer system with 57-bit virtual addressing using multi-level tree-structured page

tables with L levels for virtual to physical address translation. The page size is 4 KB (1 KB = 1024 B) and a page table entry at any of the levels occupies 8 bytes.

The value of L is _____.

4. [MCQ] [GATE-2022 : 2M]

Which one of the following statements is FALSE?

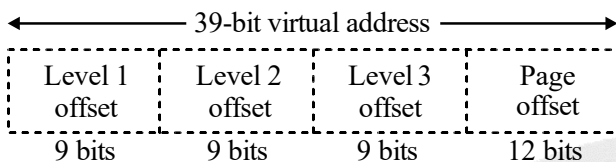
- (a) The TLB performs an associative search in parallel on all its valid entries using page number of incoming virtual address.
- (b) If the virtual address of a word given by CPU has a TLB hit, but the subsequent search for the word results in a cache miss, then the word will always be present in the main memory.
- (c) The memory access time using a given inverted page table is always same for all incoming virtual addresses.
- (d) In a system that uses hashed page tables, if two distinct virtual addresses V1 and V2 map to the same value while hashing, then the memory access time of these addresses will not be the same

5. [MSQ] [GATE-2021 : 1M]

In the context of operating systems, which of the following statements is/are correct with respect to paging?

- (a) Paging helps solve the issue of external fragmentation
- (b) Page size has no impact on internal fragmentation
- (c) Paging incurs memory overheads
- (d) Multi-level paging is necessary to support pages of different sizes

6. [NAT] [GATE-2021 : 2M]
Consider a three-level page table to translate a 39-bit virtual address to a physical address as shown below. The page size is 4KB ($1\text{KB} = 2^{10}$ bytes) and page table entry size at every level is 8 bytes. A process P is currently using 2GB ($1\text{GB} = 2^{30}$ bytes) virtual memory which is mapped to 2GB of physical memory. The minimum amount of memory required for the page table of P across all levels is _____ KB.



7. [NAT] GATE-2016 : 2M
Consider a computer system with 40-bit virtual addressing and page size of sixteen kilobytes. If the computer system has a one-level page table per process and each page table entry requires 48 bits, then the size of the per-process page table is _____ megabytes.
8. [NAT] [GATE-2015 : 2M]
A computer system implements 8 kilobyte pages and a 32-bit physical address space. Each page table entry contains a valid bit, a dirty bit, three permission bits, and the translation. If the maximum size of the page table of a process is 24 megabytes, the length of the virtual address supported by the system is _____ bits.
9. [NAT] [GATE-2015 : 2M]
A computer system implements a 40 bit virtual address, page size of 8 kilobytes, and a 128-entry translation look-aside buffer (TLB) organized into 32 sets each having four ways. Assume that the TLB tag does not store any process id. The minimum length of the TLB tag in bits is _____

10. [NAT] [GATE-2015 : 2M]
Consider a system with byte-addressable memory, 32-bit logical addresses, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in megabytes is _____
11. [NAT] [GATE-2014 : 2M]
Consider a paging hardware with a TLB. Assume that the entire page table and all the pages are in the physical memory. It takes 10 milliseconds to search the TLB and 80 milliseconds to access the physical memory. If the TLB hit ratio is 0.6, the effective memory access time (in milliseconds) is _____
12. [MCQ] [GATE-2013: 2M]
A computer uses 46-bit virtual address, 32-bit physical address, and a three-level paged page table organization. The page table base register stores the base address of the first-level (T1), which occupies exactly one page. Each entry of T1 stores the base address of a page of the second-level table (T2). Each entry of T2 stores the base address of a page of the third-level table (T3). Each entry of T3 stores a page table entry (PTE). The PTE is 32 bit in size. The processor used in the computer has a 1 MB 16-way-set associative virtually indexed physically tagged cache the cache block size is 64 bytes.
What is the size of a page in KB in this computer?
(a) 2 (b) 4
(c) 8 (d) 16
13. [MCQ] [GATE-2009 : 2M]
The essential content(s) in each entry of a page table is/are
(a) Virtual page number
(b) Page frame number
(c) Both virtual page number and page frame number
(d) Access right information
14. [MCQ] [GATE-2009: 2M]
A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because.

- (a) it reduces the memory access time to read or write a memory location.
- (b) it helps to reduce the size of page table needed to implement the virtual address space of a process.
- (c) it is required by the translation look a side buffer.
- (d) it helps to reduce the number of page faults in page replacement algorithms.

15. [MCQ] [GATE-2008: 2M]

A processor uses 36-bit physical addresses and 32-bit virtual addresses, with a page frame size of 4 Kbytes. Each page table entry is of size 4 bytes. A three-level page table is used for virtual-to-physical address translation, where the virtual address is used as follows:

- bits 30-31 are used to index into the first level page table
- bits 21-29 are used to index into the second level page table
- bits 12-20 are used to index into the third level page table
- bits 0-11 are used as offset within the page.

The number of bits required for addressing the next level page table (or page frame) in the page table entry of the first, second and third level page tables are respectively.

- (a) 20, 20 and 20
- (b) 24, 24, and 24
- (c) 24, 24 and 20
- (d) 25, 25 and 24

16. [MCQ] [GATE-2008 : 2M]

A paging scheme uses a Translation Look-aside Buffer (TLB). A TLB-access takes 10 ns and a main memory access takes 50 ns. What is the effective access time (in ns) if the TLB hit ratio is 90% and there is no page-fault?

- (a) 54
- (b) 60
- (c) 65
- (d) 75

Virtual Memory

17. [NAT] [GATE-2023: 2M]

Consider the following two-dimensional array D in the C programming language, which is stored in row-major order:

```
int D[128][128];
```

Demand paging is used for allocating memory and each physical page frame holds 512 elements of the array D. The Least Recently Used (LRU) page-replacement policy is used by the operating system. A total of 30 physical page frames are allocated to a process which executes the following code snippet:

```
for (int i = 0; i < 128; i++)
```

```
    for (int j = 0; j < 128; j++)
```

```
        D[j][i] *= 10;
```

The number of page faults generated during the execution of this code snippet is _____.

18. [NAT] [GATE-2022 : 2M]

Consider a demand paging system with four page frames (initially empty) and LRU page replacement policy. For the following page reference string 7, 2, 7, 3, 2, 5, 3, 4, 6, 7, 7, 1, 5, 6, 1 the page fault rate, defined as the ratio of number of page faults to the number of memory accesses (rounded off to one decimal place) is _____.

19. [NAT] [GATE-2020 : 2M]

Consider a paging system that uses 1-level page table residing in main memory and a TLB for address translation. Each main memory access takes 100 ns and TLB lookup takes 20 ns. Each page transfer to/from the disk takes 5000 ns. Assume that the TLB hit ratio is 95%, page fault rate is 10%. Assume that for 20% of the total page faults, a dirty page has to be written back to disk before the required page is read from disk. TLB update time is negligible. The average memory access time in ns (round off to 1 decimal places) is _____

20. [MCQ] [GATE-2019 : 2M]

Assume that in a certain computer, the virtual addresses are 64 bits long and the physical addresses are 48 bits long. The memory is word addressable.

The page size is 8 kB and the word size is 4 bytes. The translation Look-aside Buffer (TLB) in the address translation path has 128 valid entries. At most how many distinct virtual addresses can be translated without any TLB miss?

- (a) 16×2^{10} (b) 256×2^{10}
(c) 4×2^{20} (d) 8×2^{20}

21. [MCQ] [GATE-2018 : 2M]

Consider a process executing on an operating system that uses demand paging. The average time for a memory access in the system is M units if the corresponding memory page is available in memory and D units if the memory access causes a page fault. It has been experimentally measured that the average time taken for a memory access in the process is X units.

Which one of the following is the correct expression for the page fault rate experienced by the process?

- (a) $(D - M) / (X - M)$
(b) $(X - M) / (D - M)$
(c) $(D - X) / (D - M)$
(d) $(X - M) / (D - X)$

22. [NAT] [GATE-2016 : 2M]

Consider a computer system with ten physical page frames. The system is provided with an access sequence $(a_1, a_2, \dots, a_{20}, a_1, a_2, \dots, a_{20})$, where each a_i is a distinct virtual page number. The difference in the number of page faults between the last-in-first-out page replacement policy and the optimal page replacement policy is _____

23. [MCQ] [GATE-2011 : 2M]

Let the page fault service time be 10 ms in a computer with average memory access time being 20 ns. If one-page fault is generated for every 10^6 memory accesses, what is the effective access time for the memory?

- (a) 21 ns (b) 30 ns
(c) 23 ns (d) 35 ns

24. [MCQ] [GATE-2008 : 2M]

Match the following flag bits used in the context of virtual memory management on the List-I (Name of the bit) with the different purposes on the List-II (Purpose) of the table below.

List-I	List-II
I. Dirty	a. Page initialization
II. R/W	b. Write-back policy
III. Reference	c. Page protection
IV. Valid	d. Page replacement policy

Codes:

- (a) I-d, II-a, III-b, IV-c (b) I-b, II-c, III-a, IV-d
(c) I-c, II-d, III-a, IV-b (d) I-b, II-c, III-d, IV-a

Page Replacement Techniques

25. [MCQ] [GATE-2009 : 1M]

In which one of the following page replacement policies, Belady's anomaly may occur?

- (a) FIFO (b) Optimal
(c) LRU (d) MRU

26. [MCQ] [GATE-2017 : 2M]

Recall that Belady's anomaly is that the page-fault rate may increase as the number of allocated frames increases. Now, consider the following statements:

S_1 : Random page replacement algorithm (where a page chosen at random is replaced) suffers from Belady's anomaly

S_2 : LRU page replacement algorithm suffers from Belady's anomaly

Which of the following is CORRECT?

- (a) S_1 is true, S_2 is true (b) S_1 is true, S_2 is false
(c) S_1 is false, S_2 is true (d) S_1 is false, S_2 is false

27. [MCQ] [GATE-2016 : 2M]

In which one of the following page replacement algorithms it is possible for the page fault rate to increase even when the number of allocated frames increases?

- (a) LRU (Least Recently Used)
(b) OPT (Optimal Page Replacement)
(c) MRU (Most Recently Used)
(d) FIFO (First In First Out)

28. [MCQ] [GATE-2015 : 2M]
Consider a main memory with five-page frames and the following sequence of page references:
3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3
Which one of the following is true with respect to page replacement policies First-In-First Out (FIFO) and Least Recently Used (LRU)?
(a) Both incur the same number of page faults
(b) FIFO incurs 2 more-page faults than LRU
(c) LRU incurs 2 more-page faults than FIFO
(d) FIFO incurs 1 more page faults than LRU
29. [NAT] [GATE-2014 : 2M]
A system uses 3 pages frames for storing process pages in main memory. It uses the Least Recently Used (LRU) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below?
4, 7, 6, 1, 7, 6, 1, 2, 7, 2
30. [MCQ] [GATE-2014 : 2M]
A computer has twenty physical page frames which contain pages numbered 101 through 120. Now a program accesses the pages numbered 1, 2, ..., 100 in that order, and repeats the access sequence THRICE.
Which one of the following page replacement policies experiences the same number of page faults as the optimal page replacement policy for this program?
(a) Least-recently-used
(b) First-in-first-out
(c) Last-in-first-out
(d) Most-recently-used
31. [NAT] [GATE-2014 : 2M]
Assume that there are 3-page frames which are initially empty. If the page reference string is 1, 2, 3, 4, 2, 1, 5, 3, 2, 4, 6, the number of page faults using the optimal replacement policy is _____
32. [MCQ] [GATE-2012 : 2M]
Consider the virtual page reference string
1, 2, 3, 2, 4, 1, 3, 2, 4, 1
On a demand paged virtual memory system running on computer system that has main memory size of 3 page frames which are initially empty. Let LRU, FIFO and OPTIMAL denote the number of page faults under the corresponding page replacement policy. Then
(a) OPTIMAL < LRU < FIFO
(b) OPTIMAL < FIFO < LRU
(c) OPTIMAL = LRU
(d) OPTIMAL = FIFO
33. [MCQ] [GATE-2010 : 2M]
A system uses FIFO policy for page replacement. It has 4-page frames with no pages loaded to begin with. The system first accesses 100 distinct pages in some order and then accesses the same 100 pages but now in the reverse order. How many page faults will occur?
(a) 196 (b) 192
(c) 197 (d) 195
34. [MCQ] [GATE-2008 : 2M]
Assume that a main memory with only 4 pages, each of 16 bytes, is initially empty. The CPU generates the following sequence of virtual addresses and uses the Least Recently Used (LRU) page replacement policy.
0, 4, 8, 20, 24, 36, 44, 12, 68, 72, 80, 84, 28, 32, 88, 92
How many page faults does this sequence cause? What are the page numbers of the page present in the main memory at the end of the sequence?
(a) 6 and 1, 2, 3, 4
(b) 7 and 1, 2, 4, 5
(c) 8 and 1, 2, 4, 5
(d) 9 and 1, 2, 3, 5

ANSWER KEY

- | | | | |
|--------------------|-------------------|----------------------|---------------|
| 1. (c) | 2. (a) | 3. (5 to 5) | 4. (c) |
| 5. (a, c) | 6. (4108 to 4108) | 7. (384 to 384) | 8. (36 to 36) |
| 9. (22 to 22) | 10. (4 to 4) | 11. (112 to 112) | 12. (c) |
| 13. (b) | 14. (b) | 15. (d) | 16. (c) |
| 17. (4096 to 4096) | 18. (0.6 to 0.6) | 19. (154.5 to 155.5) | 20. (b) |
| 21. (b) | 22. (1 to 1) | 23. (b) | 24. (d) |
| 25. (a) | 26. (b) | 27. (d) | 28. (a) |
| 29. (6 to 6) | 30. (d) | 31. (7 to 7) | 32. (b) |
| 33. (a) | 34. (b) | | |

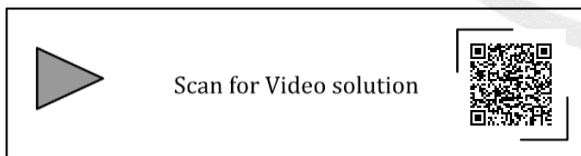
SOLUTIONS

1. (c)

Best fit always search for the smallest hole that can accommodate a request. So, hole created by best fit will always be less than or equal to the hole created by any other policy.

Worst fit always search for the largest hole to accommodate a request. If the first hole is the largest one, then worst fit and first fit both will accommodate request in the same hole.

So, we can clearly say option c is the correct option.

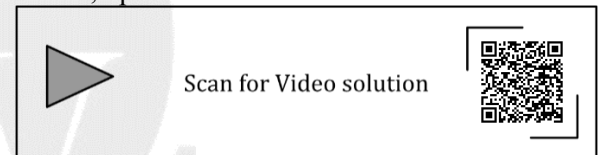


2. (a)

Here we have 6 memory partitions of sizes 200 KB, 400 KB, 600 KB, 500 KB, 300 KB, and 250 KB, and the partition allotted to four processes using best fit algorithm is as follows:

200	400	600	500	300	250
	357	491	468		210
	P1	P4	P3		P2

So, we can clearly see that partition 200KB and 300KB are empty and not allotted to any process. Hence, option A is correct.



3. (5 to 5)

L : number of paging.

Virtual Address space = 57 bits

Page Size = 4 KB

Page table entry = 8 B

Page number = $(57-12) = 45$ bits

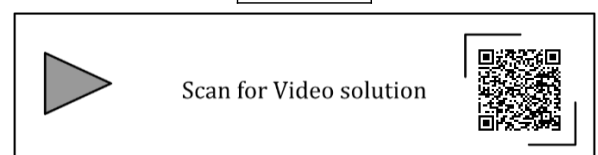
Number of entries in P.T = $\frac{4KB}{8B} = 512 = 2^9$. Means, we need 9 bits to index the page table.

level 1	level 2	level 3	level 4	level 5	offset
9	9	9	9	9	12

57 bits

So, number of levels = $9 \times L = 45$

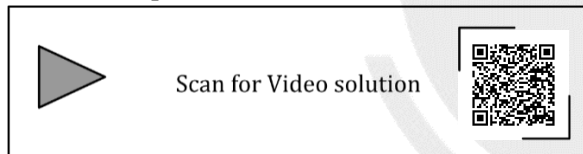
$$L = \frac{45}{9} = 5$$



4. (c)

- (a) The TLB performs an associative search in parallel on all its valid entries using page number of incoming virtual address. TRUE.
- (b) If the virtual address of a word given by CPU has a TLB hit, but the subsequent search for the word results in a cache miss, then the word will always be present in the main memory. TRUE. If the content is not in the physical address space, then it will present in main memory.
- (c) The memory access time using a given inverted page table is always same for all incoming virtual addresses. FALSE. Inverted paging has issue of time, every virtual address may present at different locations. In worst case, we may have to search entire page table.
- (d) In a system that uses hashed page tables, if two distinct virtual addresses V1 and V2 map to the same value while hashing, then the memory access time of these addresses will not be the same. TRUE. Because we have to search in linked list addresses may present in different locations.

Therefore, option C is the correct answer.



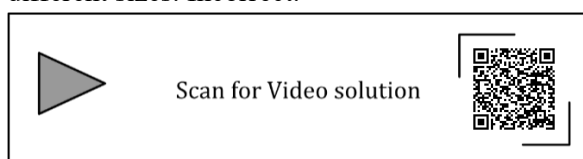
5. (a, c)

Paging helps solve the issue of external fragmentation. Correct.

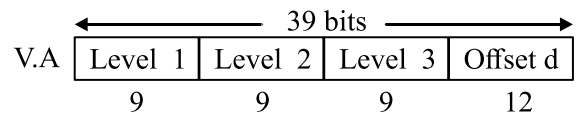
Page size has no impact on internal fragmentation. Incorrect, paging has impact on internal fragmentation

Paging incurs memory overheads. Correct. Page tables are stored in memory and are overhead to memory.

Multi-level paging is necessary to support pages of different sizes. Incorrect.



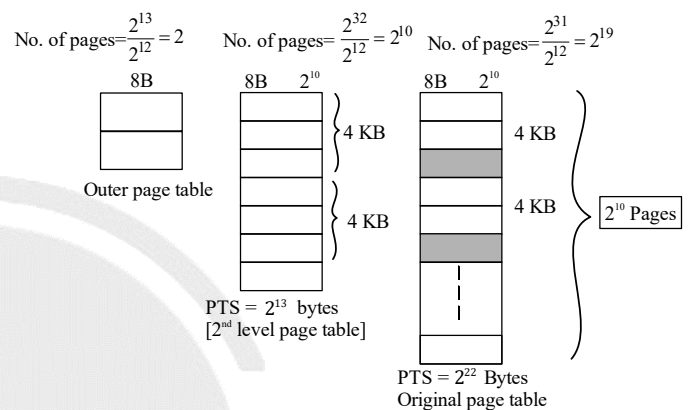
6. (4108 to 4108)



Page Size = 4 KB

Page Table Entry size = 8B

$$\frac{P}{V.M.S} = 2 \text{ GB}$$



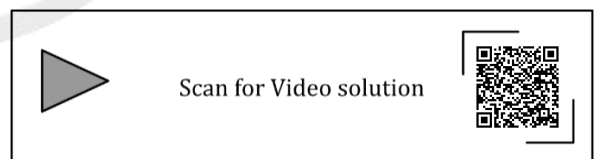
Total number of pages of PT'S needed.

= 1 k of 1 – level paging

= 2 k of 2 – level paging

= 1 k of Outer Page Table

$$= (1+2+1024) * 4 \text{ KB} = 1027 \times 4 \text{ KB} = 4108 \text{ KB}$$



7. (384 to 384)

Virtual Address = 40 bits

Page Size = 16 KB

Page Table Entry = 48 bits = 6B

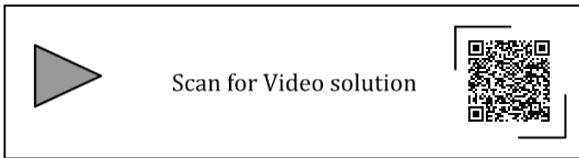
Page Table Size = N * e

$$N = \frac{2^{40}}{2^{14}} = 2^{26}$$

e = 6B

$$\therefore \text{P.T.S} = 2^{26} \times 6\text{B} = 64 \times 6 \text{ MB}$$

$$\text{P.T.S.} = 384 \text{ MB}$$



8. (36 to 36)

Given,

Page Size = 8 KB

Physical Address = 32 bits

Page table size = 24 MB

19	1	1	3
Frame No.	Valid/Invalid	Dirty	Permission

$$\text{Number of frames, } M = \frac{2^{32}}{2^{13}} = 2^{19}$$

$$\text{Page Table Entry} = 24 \text{ bits} = 3 \text{ B}$$

$$\text{P.T.S.} = N * e$$

$$24 \text{ MB} = N * 3 \text{ B}$$

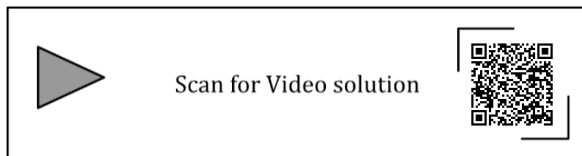
$$N = \frac{24 \text{ MB}}{3 \text{ B}} = 8 \text{ M}$$

$$N = 8 \text{ M} \Rightarrow P = 23 \text{ bits}$$

$$\therefore \text{V.A.} \begin{array}{|c|c|} \hline \text{P} & \text{d} \\ \hline \end{array}$$

$$\begin{array}{c} \xleftarrow{23} \quad \xrightarrow{13} \\ \hline 36 \text{ bits} \end{array}$$

So, the length of the virtual address is 36 bits.



9. (22 to 22)

Given,

Virtual Address = 40 bits

Page Size = 8 KB

TLB lines = 128

Sets = 32 sets

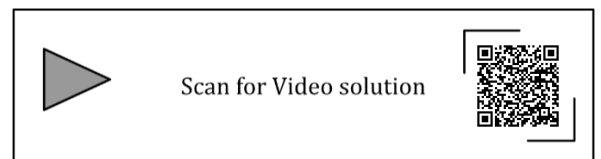
TLB Tag bits = ?

Page offset (d) = 13 bits

Set offset = 5 bit

$$\text{TLB} = \text{Tag bit} = 40 - (13 + 5) = 40 - 18 = 22 \text{ bits}$$

So, the minimum length of the TLB tag is 22 bits.



10. (4 to 4)

Logical Address space = 32 bits

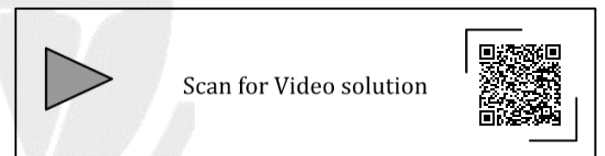
Page Size = 4 KB

Page Table Entry = 4B

Page Table Size = N * e

$$N = \frac{2^{32}}{2^{12}} = 2^{20}$$

$$\text{P.T.S.} = 1 \text{ M} * 4 \text{ B} = 4 \text{ MB}$$



11. (112 to 112)

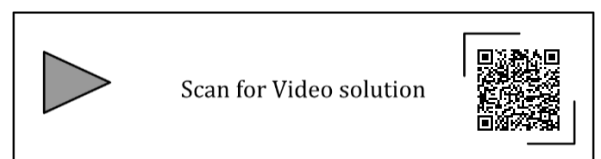
TLB access time (c) = 10 ms

Main Memory access time (m) = 80 ms

TLB Hit ratio (x) = 0.6

$$\begin{aligned} \text{EMAT} &= x(c + m) + (1 - x)(c + 2m) \\ &= 0.6(10 + 80) + 0.4(10 + 160) \\ &= 0.6 \times 90 + 0.4 \times 170 \end{aligned}$$

$$\text{EMAT} = 54 + 64 = 122 \text{ ms}$$



12. (c)

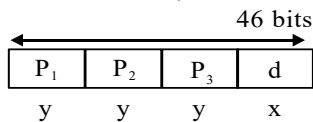
Page size = ?

Given,

Outer Page Table fit in recently one page

Virtual Address = 46 bits

Page Table Base Register = 4 B

Let P.S. = 2^x bytes

$$3y + x = 46 \dots (i)$$

OPT size = $(2^y) * 4 \text{ B} = 2 \text{ bytes}$

$$\therefore 2^{y+2} = 2^x \Rightarrow x = y + 2$$

$$3y + y + 2 = 46$$

$$4y = 44$$

$$\therefore y = 11$$

$$x = 13$$

Therefore,

$$\therefore \text{Page size} = 2^{13} = 8\text{KB}$$



Scan for Video solution



13. (b)

The essential content(s) in each entry of a page table is page frame number. Virtual page number is not stored in page table, so option A and C are clearly eliminated. Access right information is stored in page table but it is not an essential content. Therefore, option B is the correct answer.



Scan for Video solution



14. (b)

A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because it helps to reduce the size of page table needed to implement the virtual address space of a process. The main

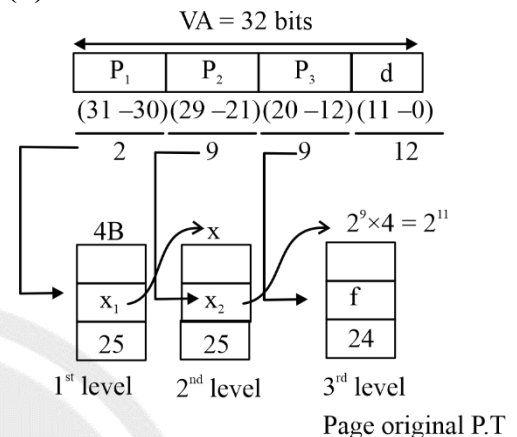
purpose of multilevel paging is to reduce the space overhead page table size overhead.



Scan for Video solution



15. (d)



P.A 36 bits

$$M = \frac{2^{36}}{2^{12}} = 2^{24}$$

F = 24

$$\frac{2^{36}}{2^{12}} = 2^{25}$$



Scan for Video solution



16. (c)

Given,

Hit ratio (x) = 90%

TLB access time (c) = 10

Main memory access time (m) = 50

So,

$$\begin{aligned} \text{EMAT} &= x(c + m) + (1-x)(c + 2m) \\ &= 0.9(60) + 0.1(10+100) \\ &= 54 + 11 = 65 \end{aligned}$$



Scan for Video solution



17. (4096 to 4096)

Each value i causes 32 Page Faults

→ matrix is stored in R.M.O

→ matrix size int D[128] [128]

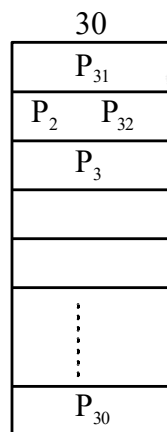
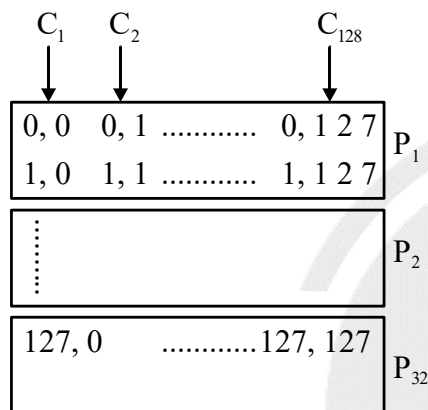
→ Page Size = 512

→ Number of frames = 30

→ Page replacement technique is LRU.

Number of pages needed to store the matrix

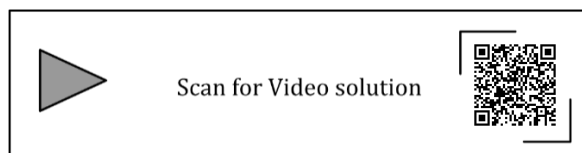
$$= \frac{128 \times 128}{512} = \frac{2^{14}}{2^9} = 2^5 = 32 \text{ pages}$$



Frames

→ each value of ' i ' causes 32 Page Faults

→ All value of i (128) causes a total of $128 \times 32 = 2^{12} = 4096$ Page Faults

**18. (0.6 to 0.6)**

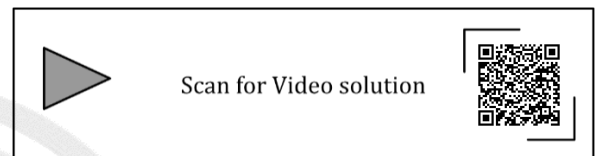
5	7	4
2	6	
3	1	
5	7	

4 = frames

Number of Page faults = 9

Total references = 15

Page fault ratio = $\frac{9}{15} = 0.6$

**19. (154.5 to 155.5)**

Given,

MMAT = 100ns;

TLBAT = 20 ns;

Disk R/W = 5000ns;

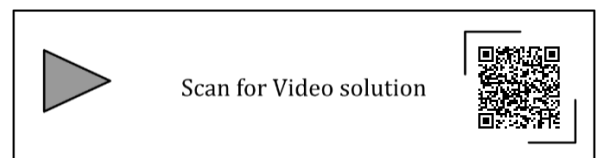
TLB Hit ratio = 95%;

Page fault rate = 10%;

Dirty page 20%

Average MAT = $0.95 (20 + 100) \text{ ns} + 0.05 [20 \text{ ns} + 0.9 (100 \text{ ns} + 100 \text{ ns}) + 0.1 (100 \text{ ns} + 0.2 (5000 + 500)) \text{ ns} + 0.8 (500) \text{ ns}]$

Average MAT = 154.5ns

**20. (b)**

Given,

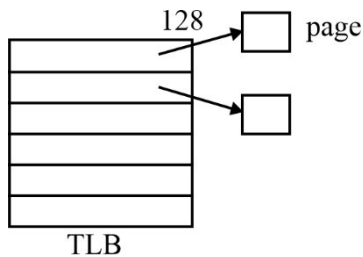
Page Size = 8KB = 2^{11}

Word Size = 4 KB

Virtual Memory Size = 2^{64} words

Number of pages possible = 2^{53}

Page offset = 64 - 53 = 11 bits



TLB contains 128 or 2^7 distinct page numbers.
One-page hit implies 2^{11} distinct virtual address hits.

So, Number of V.A $\rightarrow$ P.A = $128 \times 2k = 256 \times 2^{10}$

Therefore, option B is the correct answer.



Scan for Video solution



21. (b)

Main Memory Access Time = M

Page Fault Service Time = D

Effective Memory Access Time = X

Page Fault rate = 'P' = ?

EMAT = $(1 - P) \text{MMAT} + P * \text{PFST}$

$$X = (1 - P) M + P * D$$

$$= M - M.P + D.P$$

$$X - M = P (D - M)$$

$$\therefore P = \frac{X - M}{D - M}$$



Scan for Video solution



22. (1 to 1)

10 frames;

Reference string $\langle a_1, a_2, a_3 \dots a_{20}, a_1, a_2, a_3, a_4 \dots a_9,$

$a_{10} a_{11}, a_{12} \dots a_{20} \rangle$

I. L.I.F.O

- (a) $a_1 \dots a_{10} \rightarrow 10$ PF's
 - (b) $a_{11} \dots a_{20} \rightarrow 10$ PF's
 - (c) $a_1 \dots a_9 \rightarrow X$
 - (d) $a_{10} \dots a_{20} \rightarrow 11$
- P.F. 31

10 Frames

a_1
a_9
a_{20} a_{10}

II. optimal

- (a) $a_1 \dots a_{10} \rightarrow 10$ PF's
 - (b) $a_{11} \dots a_{20} \rightarrow 10$ PF's
 - (c) $a_1 \dots a_9 \rightarrow X$
 - (d) $a_{10} \dots a_{19} \rightarrow 10$ PF's
 - (e) $a_{20} \dots$ No. P.F.
- Total 30

10 Frames

a_1
a_2
a_9
a_{20}

Using LIFO, first 20 pages will result in page faults and next 9 pages will be page hit and then next 11 pages will be page faults.

Using optimal page replacement algorithm, first 20 pages will result in page faults and next 9 pages will be page hit and then next 10-page faults followed by page hit for last page.



Scan for Video solution



23. (b)

Memory access time (x) = 20ns

Page fault rate (p) = $1/10^6$

Page fault service time (y) = 10ms = 10×10^{-3} ns

Effective Memory Access Time = $(1 - p) * x + p * y$

$$\begin{aligned} \text{E.M.A.T} &= \left(1 - \frac{1}{10^6}\right) \times 20\text{ns} + \frac{1}{10^6} \times 10 \times 10^{-3} \\ &= 1 \times 20\text{ns} + 10 \times 10^{-9} \\ &= 20 + 10 = 30\text{ns} \end{aligned}$$

So, option B is the correct answer,



Scan for Video solution



24. (d)

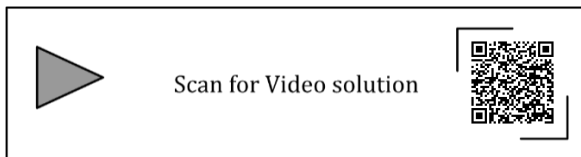
Dirty bit is used in write-back policy.

Read/write bit is used for page protection.

Reference bit is used for page replacement policies.

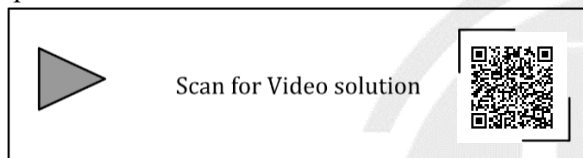
Valid bit is used for page initialization.

So, option D is the correct answer.



25. (a)

FIFO (First in First out) page replacement algorithm suffers from Belady's Anomaly. So, option A is correct.

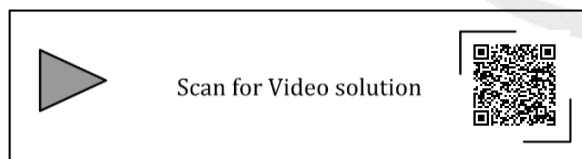


26. (b)

S1: Random page replacement may suffer from belady's anomaly when it starts behaving like FIFO page replacement algorithm.

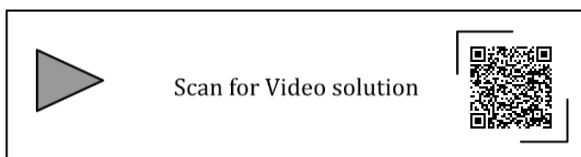
S2: LRU does not suffers from belady's anomaly. LRU is a stack algorithm and stack algorithm does not suffers from belady's anomaly.

So, option B is correct.



27. (d)

FIFO page replacement algorithm can result in increasing page fault rate even when the number of allocated frames are decreasing.



28. (a)

Reference string:

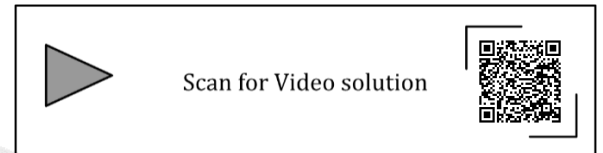
3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3

Page fault using FIFO and LRU page replacement algorithm:

FIFO		LRU	
9 PF's	3 6	3	
	3 3	3 6	
	3 8	3 6 1	
	3 2	3	
	1	3 2	

5 – Frames

5 – Frames



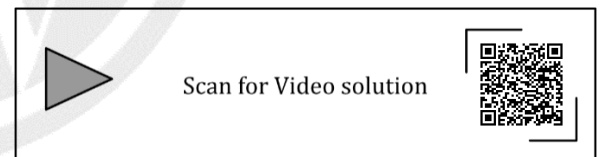
29. (6 to 6)

Reference string: 4, 7, 6, 1, 7, 6, 1, 2, 7, 2

4	7	6	1	7	6	1	2	7	2
4	4	4	1	1	1	1	1	1	1
	7	7	7	7	7	7	2	2	2
		6	6	6	6	6	6	7	7

PF PF PF PF PH PH PH PF PF PH

PF is page fault and PH is page hit, so total 6 number of page faults.



30. (d)

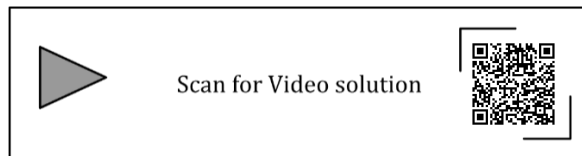
In optimal page replacement policy, the page which will next occur farthest in the future will be swapped out from the memory.

It is given that, computer has 20-page frames and initially the pages are numbered from 1 to 20. And the program accesses the pages numbered 1, 2, ..., 100 and repeats the sequence THRICE.

The first 20 page access (i.e. 1 to 20) will cause page fault. Now, when page 21 will arrive it will swap out page 20 as 20 is going to be used farthest in future, similarly when 22 will arrive so 21 will be swapped out and so on.

So, the given algorithm is working as most-recently used. Hence, option D is the correct answer.

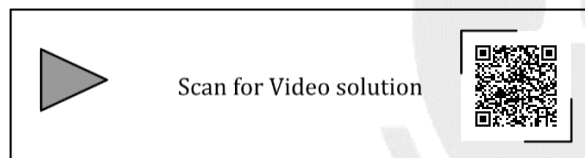
1	2	3	4	2	1	5	3	2	4	6
1	1	1	1	1	1	5	3	3	3	6
	2	2	2	2	2	2	2	2	2	2
		3	4	4	4	4	4	4	4	4
PF	PF	PF	PF	PH	PH	PF	PF	PH	PH	PF



31. (7 to 7)

Reference string: 1, 2, 3, 4, 2, 1, 5, 3, 2, 4, 6

1	2	3	4	2	1	5	3	2	4	6
1	1	1	1	1	1	5	3	3	3	6
	2	2	2	2	2	2	2	2	2	2
		3	4	4	4	4	4	4	4	4
PF	PF	PF	PF	PH	PH	PF	PF	PH	PH	PF



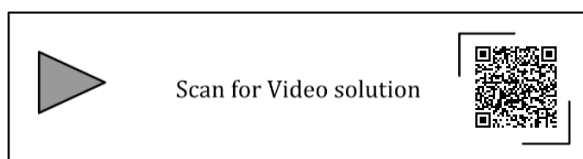
32. (b)

The page faults using FIFO, optimal and LRU are as follows:

① FIFO	② Optimal	③ L.R.U																		
6 PF'S	5 PF'S	9 PF'S																		
<table> <tr><td>1</td><td>4</td></tr> <tr><td>2</td><td>1</td></tr> <tr><td>3</td><td>2</td></tr> </table>	1	4	2	1	3	2	<table> <tr><td>1</td><td></td></tr> <tr><td>2</td><td>4</td></tr> <tr><td>3</td><td>2</td></tr> </table>	1		2	4	3	2	<table> <tr><td>1</td><td>2</td></tr> <tr><td>3</td><td>1</td></tr> <tr><td>4</td><td>4</td></tr> </table>	1	2	3	1	4	4
1	4																			
2	1																			
3	2																			
1																				
2	4																			
3	2																			
1	2																			
3	1																			
4	4																			
3 Frames	3 Frames	3 Frames																		

Optimal < FIFO < LRU

So, option B is correct.



33. (a)

Given that the system has 4 page frames.

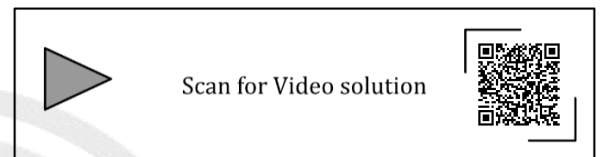
Initially for page number 1 to 100 it will cause page faults, then in reverse order for page number 100, 99, 98, and 97 it will be a page hit and then again page faults for page 96 to page 1.

[1, 2, ..., 100, 100, ..., 2, 1]

So, $2n - 4$, we are accessing n pages for 2 times and only 4 page hits are there. Here, N equals to 100.

$$200 - 4 = 196.$$

Therefore, option A is correct.



34. (b)

Page Size = 16 bytes

Number of frames, $M = 4$;

Virtual Addresses:

0, 4, 8, 20, 24, 36, 44, 12, 68, 72, 80, 84, 28, 32, 88, 92

Reference String corresponding to given virtual addresses:

<0, 0, 0, 1, 1, 2, 2, 0, 4, 4, 5, 5, 1, 2, 5, 5>

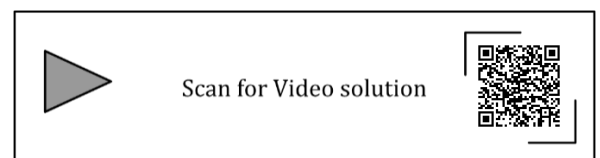
<0, 1, 2, 0, 4, 5, 1, 2, 5>

LR U :	0 2
	1 5
	2 1
	4

Number of P.F's : 7

The page numbers of the page present in the main memory at the end of the sequence is 1, 2, 4, 5.

So, option B is the correct answer.



File System

1. [NAT] [GATE-2022 : 2M]

Consider two files systems A and B , that use contiguous allocation and linked allocation, respectively. A file of size 100 blocks is already stored in A and also in B . Now, consider inserting a new block in the middle of the file (between 50th and 51st block), whose data is already available in the memory. Assume that there are enough free blocks at the end of the file and that the file control blocks are already in memory. Let the number of disk accesses required to insert a block in the middle of the file in A and B are n_A and n_B , respectively, then the value of $n_A + n_B$ is _____.

2. [MSQ] [GATE-2021 : 1M]

Consider a linear list-based directory implementation in a file system. Each directory is a list of nodes, where each node contains the file name along with the file metadata, such as the list of pointers to the data blocks. Consider a given directory foo.

Which of the following operations will necessarily require a full scan of foo for successful completion?

- (a) Creation of a new file in foo
- (b) Deletion of an existing file from foo
- (c) Renaming of an existing file in foo
- (d) Opening of an existing file in foo

Disk Scheduling Algorithm

3. [MCQ] [GATE-2020 : 2M]

Consider the following five disk access requests of the form (request id, cylinder number that are present in the disk scheduler queue at a given time.

(P, 155), (Q, 85), (R, 110), (S, 30), (T, 115)

Assume the head is positioned at cylinder 100. The scheduler follows Shortest Seek Time First scheduling to service the requests,

Which one of the following statements is FALSE?

- (a) R is serviced before P
- (b) Q is serviced after S but before T.
- (c) T is serviced before P.
- (d) The head reverses its direction of movement between servicing of Q and P

4. [NAT] [GATE-2018 : 2M]

Consider a storage disk with 4 platters (numbered as 0, 1, 2 and 3), 200 cylinders (numbered as 0, 1, ..., 199) and 256 sectors per track (numbered as 0, 1, ..., 255). The following 6 disk requests of the form [sector number, cylinder number, platter number] are received by the disk controller at the same time:

[120, 72, 2], [180, 134, 1], [60, 20, 0],

[212, 86, 3], [56, 116, 2], [118, 16, 1]

Currently the head is positioned at sector number 100 of cylinder 80 and is moving towards higher cylinder numbers. The average power dissipation in moving the head over 100 cylinders is 20 milliwatts and for reversing the direction of the head movement once is 15 milliwatts. Power dissipation associated with rotational latency and switching of head between different platters is negligible.

The total power consumption in milliwatts to satisfy all of the above disk requests using the Shortest Seek Time First disk scheduling algorithm is

5. [NAT] [GATE-2016 : 2M]

Consider a disk queue with requests for I/O to blocks on cylinders 47, 38, 121, 191, 87, 11, 92, 10. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 63, moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is ____

6. [MCQ] [GATE-2015 : 2M]

Suppose the following disk request sequence (track numbers) for a disk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80, 25, 70. Assume that the initial position of the R/W head is on track 50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is ____ tracks.

- (a) 8 (b) 9
(c) 10 (d) 11

7. [NAT] [GATE-2014 : 2M]

Suppose a disk has 201 cylinders, numbered from 0 to 200. At some time the disk arm is at cylinder 100, and there is a queue of disk access requests for cylinders 30, 85, 90, 100, 105, 110, 135 and 145. If Shortest-Seek Time First (SSTF) is being used for scheduling the disk access, the request for cylinder 90 is serviced after servicing ____ number of requests.

8. [MCQ] [GATE-2009 : 2M]

A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because.

- (a) it reduces the memory access time to read or write a memory location.
(b) it helps to reduce the size of page table needed to implement the virtual address space of a process.

- (c) it is required by the translation look a side buffer.
(d) it helps to reduce the number of page faults in page replacement algorithms.

9. [MCQ] [GATE-2009 : 2M]

Consider a disk system with 100 cylinders. The requests to access the cylinders occur in following sequence:

4, 34, 10, 7, 19, 73, 2, 15, 6, 20

Assuming that the head is currently at cylinder 50, what is the time taken to satisfy all requests if it takes 1 ms to move from one cylinder to adjacent one and shortest seek time first policy is used?

- (a) 95 ms (b) 119 ms
(c) 233 ms (d) 276 ms

File Allocation Methods

10. [NAT] [GATE-2019 : 2M]

The index node (inode) of a Unix-like file system has 12 direct, one single-indirect and one double-indirect pointers. The disk block size is 4 kB, and the disk block address is 32-bits long. The maximum possible file size is (rounded off to 1 decimal place) ____ GB.

11. [MCQ] [GATE-2017 : 1M]

In a file allocation system, which of the following allocation scheme(s) can be used if no external fragmentation is allowed?

- I. Contiguous II. Linked
III. Indexed
(a) I and III only (b) II only
(c) III only (d) II and III only

12. [NAT] [GATE-2014 : 1M]

A FAT (file allocation table) based file system is being used and the total overhead of each entry in the FAT is 4 bytes in size. Given a 100×10^6 bytes disk on which the file system is stored and data block size is 10^3 bytes, the maximum size of a file that can be stored on this disk in units of 10^6 bytes is ____

13. [MCQ] [GATE-2012 : 2M]

A file system with 300 GByte disk uses a file descriptor with 8 direct block addresses, 1 indirect block address and 1 doubly indirect block address. The size of each disk block is 128 Bytes and the size of each disk block address is 8 Bytes. The maximum possible file size in this file system is

- (a) 3 KBytes
- (b) 35 KBytes
- (c) 280 Kbytes
- (d) dependent on the size of the disk.

14. [MCQ] [GATE-2008: 2M]

The data block of a very large file in the Unix file system are allocated using.

- (a) Contiguous allocation
- (b) Linked allocation
- (c) Indexed allocation
- (d) An extension of indexed allocation

□□□



ANSWER KEY

- | | | | |
|-----------------|--------------|-------------|----------------------|
| 1. (153 to 153) | 2. (a, c) | 3. (b) | 4. (85 to 85) |
| 5. (346 to 346) | 6. (c) | 7. (3 to 3) | 8. (b) |
| 9. (b) | 10. (4 to 4) | 11. (d) | 12. (99.55 to 99.65) |
| 13. (b) | 14. (d) | | |

SOLUTIONS

1. (153 to 153)

F_A : Contiguous allocation : 100 blocks (Array)

F_B : Linked allocation: 100 blocks

We have over total data if 100

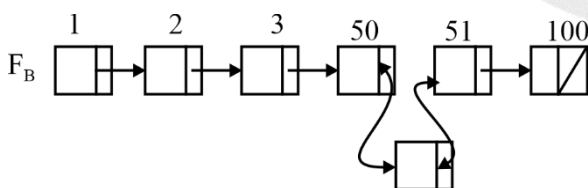
I) F_A : $50 \times 2 = 100 + 1 = 101$ (Total disk accesses)

II.

Contiguous Allocation can directly traverse to the 50th element as it is already given the file control blocks is already present.

Now, we need 50 operations to read the rest of the 50 elements and another 50 operations to write those 50 blocks and 1 operation is needed to write the new block.

Therefore, total operations in this case = $50 + 50 + 1$.



Linked Allocation needs to traverse to the 50th element.

Now, simply change the pointers just like the Linked list where Adding element in the middle of it.

So, here it takes 50 operations to read the first 50 elements and 2 operations to change the pointer of the 50th block and a new block. So, 52 operations in this case.

$$F_B = 50 + 1 + 1 = 52 \text{ } n_B$$

$$\therefore n_A + n_B = 101 + 52 = 153$$



Scan for Video solution



2. (a, c)

Creating a new file in foo will definitely require a full scan, so that we can check that the new file is not conflicting with any other already existing file. Also, renaming of a existing file will require a full scan, to ensure that the updated name is not conflicting with any other existing file name.

Deleting or opening an existing file, do not need a full scan.



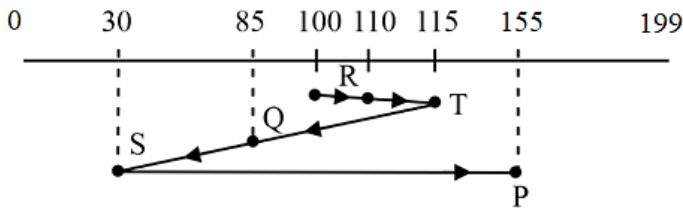
Scan for Video solution



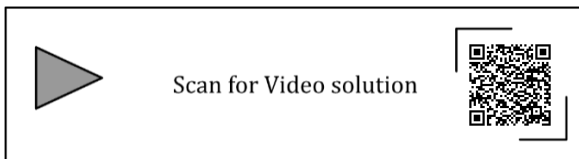
3. (b)

Request: $\langle (P, 155), (Q, 85), (R, 110), (S, 30), (T, 115) \rangle$

Initially head at cylinder 100, scheduling using SSTF:



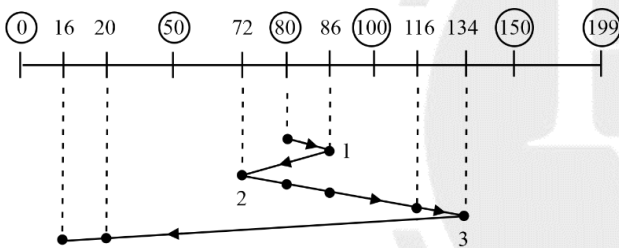
From the above diagram we can see that: Q is serviced after S but before T is wrong. Q is serviced before S and after T. So, option B is the answer.



4. (85 to 85)

Request: $\langle 72, 134, 20, 86, 116, 16 \rangle$

Initially the head is positioned at 100, and scheduling using S.S.T.F:



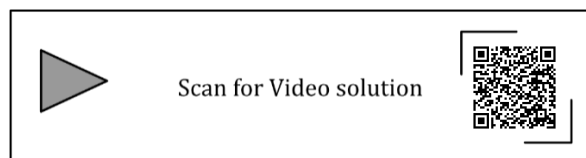
Total seeks: 200

Power consumption for seek movements: $2 \times 20 = 40$ mw

No. of direction changes 3

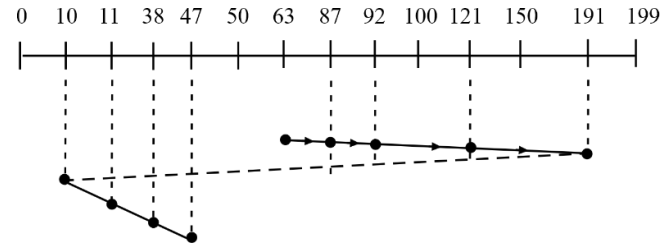
Power consumption for R/W head direction changing: $3 \times 15 = 45$ mw

Total power consumption : $40 + 45 = 85$ mw



5. (346 to 346)

Initially the head is at 63, and moving toward larger cylinder number, scheduling using C-LOOK:

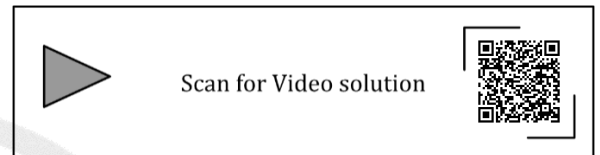


The total head movement incurred:

$$(191 - 63) + (191 - 10) + (47 - 10)$$

$$= 128 + 181 + 37$$

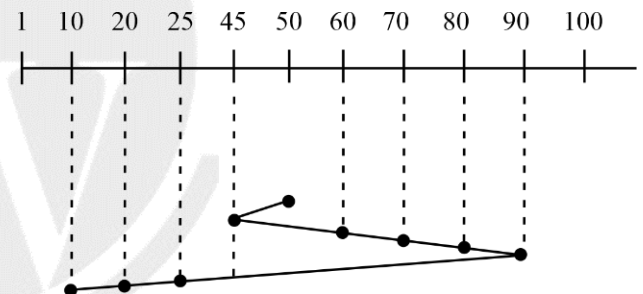
$$= 346$$



6. (c)

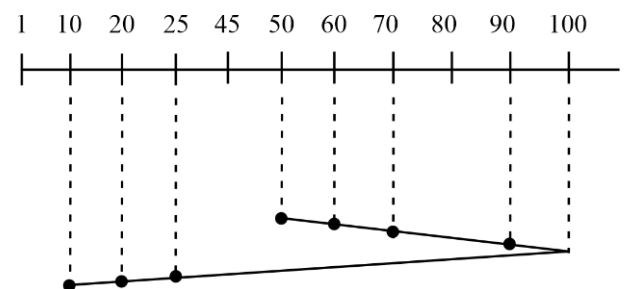
Requests: $\langle 45, 20, 90, 10, 50, 60, 80, 25, 70 \rangle$

Scheduling using S.S.T.F:



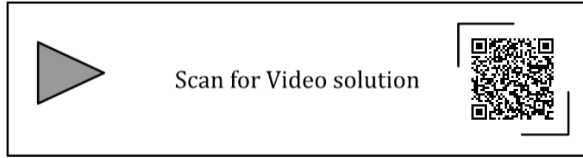
$$\text{Number of seeks: } (90 - 45) + (50 - 45) + (90 - 10) = 45 + 5 + 80 = 130$$

Scheduling using SCAN:



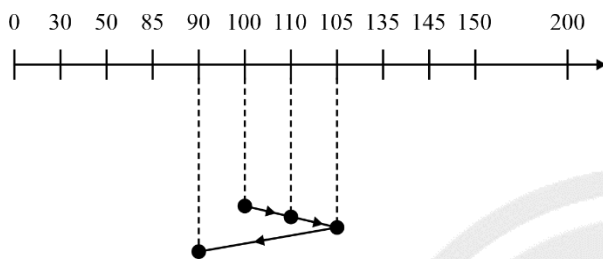
$$\text{No of seeks: } (100 - 50) + (100 - 10) = 50 + 90 = 140$$

Additional seeks = $140 - 130 = 10$
So, option C is the correct answer.

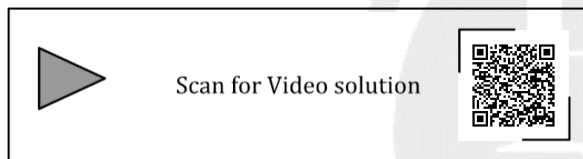


7. (3 to 3)

Scheduling using SSTF:

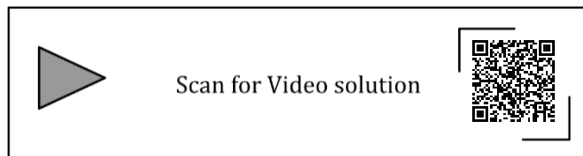


Before servicing 90, we have serviced, 100, 105, 110. So, 3 is the correct answer..



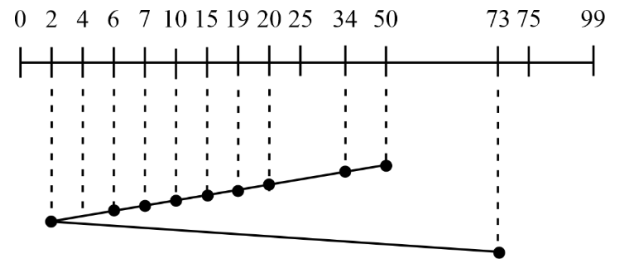
8. (b)

A multilevel page table is preferred in comparison to a single level page table for translating virtual address to physical address because it helps to reduce the size of page table needed to implement the virtual address space of a process. The main purpose of multilevel paging is to reduce the space overhead page table size overhead.



9. (b)

Request: $\langle 4, 34, 10, 7, 19, 73, 2, 15, 6, 20 \rangle$
Initially head is at 50, Scheduling using SSTF:

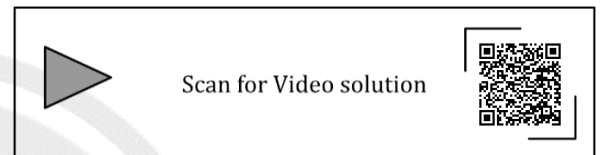


Total seeks: $(73 - 2) + (50 - 2) = 71 + 48 = 119$.

Time to go from one cylinder to next adjacent cylinder = 1ms

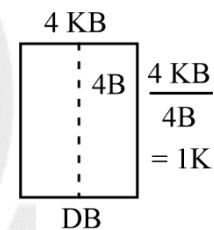
Total seek time = $119 \times 1 \text{ ms} = 119 \text{ ms}$

So, option B is the correct answer.



10. (4 to 4)

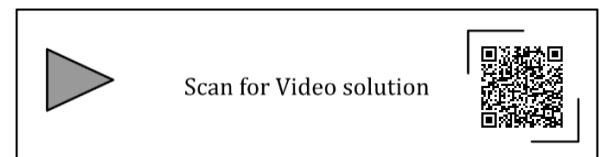
DBS = 4KB; DBA = 32 bits (4B)



1K disk block addresses can be stored.

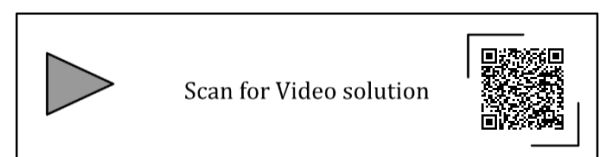
Maximum file size:

$$(12 * 4KB + 1K * 4KB + 1K * 1K * 4KB) = 48KB + 4MB + 4GB = 0.004 \text{ GB} + 4GB = 4.0 \text{ GB}$$

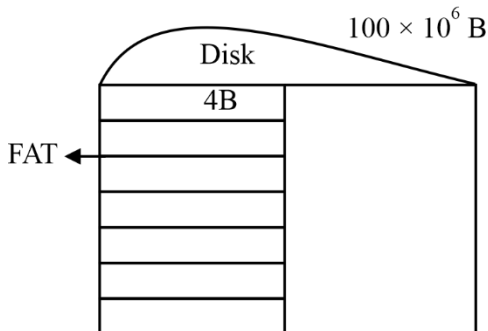


11. (d)

Linked and indexed allocation can be used when no external fragmentation is allowed.



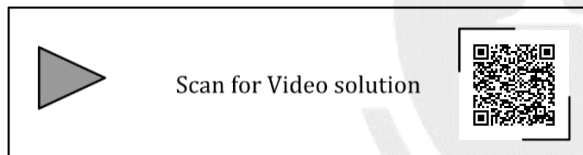
12. (99.55 to 99.65)



$$\text{No. of blocks on disk} = \frac{100 \times 10^6}{10^3} = 100 \times 10^3$$

$$\begin{aligned} \text{FAT size} &= 100 \times 10^3 \times 4\text{B} \\ &= 4 \times 100 \times 10^3 \\ &= 4 \times 10^5 \\ &= 0.4 \times 10^6 \end{aligned}$$

$$\begin{aligned} \text{Maximum file size} &= \text{Disk size} - \text{FAT size} \\ &= 100 \times 10^6 - 0.4 \times 10^6 \\ &= 100 \text{ MB} - 0.4 \text{ MB} \\ &= 99.6 \text{ MB} \end{aligned}$$



13. (b)

Disk Block Size = 128B

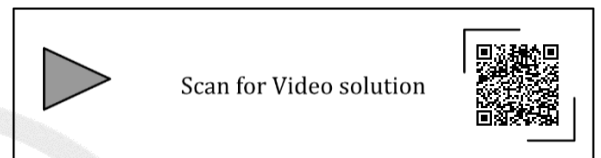
Disk Block Address = 8B

In one block we can store:

$$\frac{128}{8} = \frac{2^7}{2^3} = 2^4 = 16 \text{ disk block pointer.}$$

$$\begin{aligned} \text{Maximum file size} &= 8 \times 128\text{B} + 16 \times 128\text{B} \\ &\quad + 16 \times 16 \times 128\text{B} \\ &= 1\text{KB} + 2\text{KB} + 32\text{KB} = 35\text{KB} \end{aligned}$$

So, option B is the correct answer.



14. (d)

The data block of a very large file in the Unix file system are allocated using an extension of indexed allocation

